

INTERNET OF THINGS EMPOWERED SMART GREEN HOUSE FARMING

*A Project Report submitted
in partial fulfillment of the requirements for the award of the degree
of BACHELOR OF TECHNOLOGY*

in

ELECTRONICS AND COMMUNICATION ENGINEERING

by

V.TEJA(19335A0415)

T.TEJOVENKATESH(18331A04F6)

R.JAICHANDRA (18331A04C3)

Under the esteemed guidance of

**Smt. U.N. SUBHADRA DEVI, M Tech
Assistant Professor**



**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
MAHARAJ VIJAYARAM GAJAPATHI RAJ COLLEGE OF ENGINEERING (A)**

**Approved by AICTE, New Delhi Permanently Affiliated to JNTU, Kakinada. Re-accredited by
NBA & NAAC with 'B+' Grade Listed u/s 2(f) and 12(B) of the UGC Act 1956 Vijay Nagar
Campus, Vizianagaram, Andhra Pradesh- 535005**

2018-2022

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
MAHARAJ VIJAYARAM GAJAPATHI RAJ COLLEGE OF ENGINEERING (A)



CERTIFICATE

This is to certify that the project entitled “**INTERNET OF THINGS EMPOWERED SMART GREEN HOUSE FARMING**” being submitted by **V.TEJA(19335A0415), T.TEJO VENKATESH(18331A04F6), R.JAI CHANDRA(18331A04C3)**, in partial fulfillment for award of the degree of **Bachelor of Technology** in **Electronics and Communication Engineering** is a record of bonafide work done by them under my supervision during the academic year 2021-2022.

PROJECT GUIDE

Mrs.U.N.SUBHADRA DEVI ,M-TECH

Assistant Professor,
Department of ECE,

M.V.G.R College of Engineering (A)
Vizianagaram.

HEAD OF THE DEPARTMENT

Dr.M.SUNIL PRAKASH, M.E, Ph.D.

Professor,
Department of ECE,

M.V.G.RCollege of Engineering
Vizianagaram.

ACKNOWLEDGEMENT

With reference and humility, we wish to express our deep sense of gratitude to **Smt. U.N. SUBHADRA DEVI** madam, Assistant Professor, Department of Electronics and communication engineering for her wholehearted co-operation, unfailing inspiration and benevolent guidance. Throughout the project work, her valuable suggestions support and constant encouragement has given a right direction and shape to our learning. Really, we are indebted to her for her excellent and enlightened guidance.

We are profoundly thankful to **Dr.M.Sunil Prakash**, Professor and Head of the Department, for his cooperation and encouragement.

We thank **Dr.K.V.L.Raju**, Principal, for extending his utmost support and cooperation in providing all the provisions for the successful completion of the project.

With great solemnity and sincerity, we offer our profuse thanks to our management, **MANSAS** for providing all the resources to complete our project successfully.

We sincerely thank to all the staff of the Department of Electronics and Communication Engineering for their sustained help in our pursuits.

We thank all those who contributed directly or indirectly in successfully carrying out this work.

V.TEJA(19335A0415)

T.TEJO VENKATESH(18331A04F6)

R. JAI CHANDRA(18331A04C3)

Mission and Vision of the Institute

Institute Vision

Maharaj Vijayaram Gajapathi Raj College of Engineering strives to become a centre par excellence for technical education where aspiring students can be transformed into skilled and well-rounded professionals with strong understanding of fundamentals, a flair for responsible innovation in engineering practical solutions applying the fundamentals, and confidence and poise to meet the challenges in their chosen professional spheres.

Institute Mission

The management believes imparting quality education in an atmosphere that motivates learning as a social obligation which we owe to the students, their parents/guardians and society at large and hence the effort is to leave no stone unturned in providing the same with all sincerity.

Department of ECE Vision

To evolve as a centre of excellence in Electronics and Communication Engineering in the country training students in contemporary technologies meeting the standards of global industry and to develop them into skillful engineers who are technologically superior and ethically strong with knowledge of core as well as inter disciplinary domains.

Department of ECE Mission

The Department of Electronics and Communication Engineering focuses in imparting quality education for students to make them compete internationally, able to effectively address societal challenges, develop intellectual qualities and social responsibility.

PROGRAM OUTCOMES (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems. 2.

Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

4. Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

5. Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

6. The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

7. Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

8. Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

9. Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

11. Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROJECT WORK

COURSE OBJECTIVES:

S.No.	COURSE OBJECTIVES
1	Students will acquire the ability to identify, describe the project, collect and analyze the data required to solve the problems using modern engineering tools and techniques.
2	Students will acquire the skills to communicate the ideas clearly and effectively in both written and oral forms.
3	Students will acquire collaborative skills through working in a team to achieve common goals.
4	Students will be able to learn on their own, reflect on their learning and take appropriate actions to improve it.

COURSE OUTCOMES MAPPING WITH POs:

S.No.	COURSE OUTCOMES	PO Mapping
1	Have the ability to identify, describe the project, collect and analyze the data required to solve the problems using modern engineering tools and techniques	High-1,2,3,4,5,9 Medium-4,7,8,10,11,12 Low-6
2	Have the ability to communicate the ideas clearly and effectively in both written and oral forms.	
3	Have the ability to work in teams to achieve common goals through collaborative skills.	
4	Have the ability to learn on their own, reflect on their on their learning and taking appropriate actions to improve it.	

ABSTRACT

The rapid change of climate, population explosion, and reduction of arable lands are calling for new approaches to ensure sustainable agriculture and food supply for the future. Greenhouse agriculture is considered to be a viable alternative and sustainable solution, which can combat the future food crisis by controlling the local environment and growing crops all year round, even in harsh outdoor conditions. However, greenhouse farms persist many challenges for efficient operation and management.

The evolving Internet of Things (IoT) technologies, which encompass the smart sensors, devices, network topologies, big data analytics, and intelligent decision is believed to be the solution in addressing the key challenges facing the greenhouse farming, such as greenhouse local climate control, crop growth monitoring, crop harvesting and etc.

This work is primarily about the improvement of current agricultural practices by using modern technologies for better yield. This work provides a model of a smart greenhouse, which helps the farmers to carry out the work in a farm automatically without the use of much manual inspection. Greenhouse, being a closed structure protects the plants from extreme weather conditions namely: wind, hailstorm, ultraviolet radiations, and insect and pest attacks.

The irrigation of agriculture field is carried out using automatic drip irrigation, which operates according to the soil moisture threshold set accordingly so as optimal amount of water is applied to the plants. Based on data from soil health card, proper amount of nitrogen, phosphorus, potassium and other minerals can be applied by using drip fertigation techniques. Proper water management tanks are constructed and they are filled with water after measuring the current water level using an ultrasonic sensor.

Plants are also provided the requisite wavelength light during the night using growing lights. Temperature and air humidity are controlled by humidity and temperature sensors and a fogger is used to control the same. A tube well is controlled using GSM module (missed call or sms).

CONTENTS

Page No

CERTIFICATE	i
ACKNOWLEDGEMENT	ii
INSTITUTE VISSION AND MISSION	iii
ABSTRACT	vi
CONTENTS	vii
LIST OF FIGURES	xi

CHAPTER 1: INTRODUCTION

- 1.1Motivation and Theoretical Overview 1
- 1.2Problem Statement 1
- 1.3Research objective 1
- 1.4Scope of Study 2
- 1.5Research Methodology 2

CHAPTER 2: INTRODUCTION TO EMBEDDED SYSTEMS

- 2.1Introduction to Embedded system
- 2.2Brief History
- 2.3Overview of Embedded System Architecture
 - 2.3.1Central Processing unit
 - 2.3.2Memory

2.3.3Input Devices

2.3.4Output Devices

2.3.5Communicating Interfaces

2.4Pheripherals

2.5Processors

2.6Reliability

CHAPTER 3: ANALYSIS OF PROJECT

3.1Overview

3.2Circuit Diagram

3.3Steps of Working

3.4Description of the circuit

CHAPTER 4: AURDINO MICROCONTROLLER

4.1Introduction

4.2Definition of Microcontroller

4.3Microcontroller vs Microprocessor

4.4Features of ATMEGA 328

4.5Audirno

4.6History of audirno

4.7Basic terminologies in aurdino

4.8Conclusion and Future scope

CHAPTER 5: Hardware Modules

5.1 RS232

5.1.1Specifications

5.1.2Asynchronous serial data bus(UART) interface

5.1.3 serial data bus

5.1.4Features

5.1.5Connection to the system

5.2GSM

5.2.1What is GSM?

5.2.2History

5.2.3GSM Frequencies

5.3 ESP8266

5.3.1 Introduction

5.3.2 What is ESP8266?

5.3.3 Pin configuration /pin diagram

5.4 Introduction to AT Commands

5.5 LM35 Temperature sensor

5.6 Relay

5.6.1 Relay driver IC ULN2003

5.7 Soil Moisture sensor

5.7.1 Sensor technology

5.8 Liquid Crystal Display

5.8.1 Pin Functions

5.8.2 LCD Screen

5.8.3 LCD Basic Commands

5.8.4 LCD Connections

5.8.5 LCD Initialisation

5.9 Resistive Humidity Sensor

5.9.1 Working of Resistive Humidity Sensor

5.10 Light Sensor

CHAPTER 6 Software Module

6.1 Software

6.2 Arduino/processing language comparison

6.3 Simulator for Arduino

6.4 Applications of Arduino

CHAPTER 7: SOURCE CODE

CHAPTER 8: RESULTS AND ANALYSIS	39
ADVANTAGES AND APPLICATIONS	47
LIMITATIONS	48
FUTURE SCOPE	49
CONCLUSION	50
REFERENCES	51

List of Figures

S.No	Figure	Page No
1.	Figure.1.1Model Diagram	
2.	Figure.2.1Circuit Diagram	
3.	Figure.2.2Communication between microprocessor and memory device	
4.	Figure.3.1Block Diagram	
5.	Figure.3.2Circuit Diagram	
6.	Figure.4.1Showing a typical microcontroller device and its different subunits	
7.	Figure.4.2Arduino UNO	
8.	Figure.4.3Arduino board layout	
9.	Figure.4.4Arduino pin diagram	
10.	Figure.4.5ACadapter	
11.	Figure.4.6Battery	
12.	Figure.4.7Arduino can run off with USB or external power supply	
13.	Figure.4.8Arduino UNO	
14.	Figure.4.9Arduino Mega 256	
15.	Figure.5.1RS232	
16.	Figure.5.2GSM network architecture	
17.	Figure.5.3Esp8266	
18.	Figure.5.4pin configuration of ESP8266	
19.	Figure.5.5Use of Max232	
20.	Figure.5.6Temperature Sensor	
21.	Figure.5.7Relay	
22.	Figure.5.8Relay circuit diagram	
23.	Figure.5.9Internal circuit diagram of relay	
24.	Figure.5.10ULN2003	
25.	Figure.5.11Soil sensor	
26.	Figure.5.12Lcd display	
27.	Figure.5.13Lcd connection	
28.	Figure.5.14Resistive humidity sensor	
29.	Figure.5.15Light sensor	
30.	Figure.6.1 A screenshot of the Arduino IDE showing the “Blink”, program, a simple beginner program	
31.	Figure.6.3Screenshot of Arduino simulator	

S.NO	TABLE	PAGE NO
1.	Table .4.1 Different Flavors of Arduino with their configuration	
2.	Table.5.1 RS232 specifications	
3.	Table.5.2UART parameters	
4.	Table.5.3 Pin configuration	
5.	Table.5.4 LCD Commands	
6.	Table.6.2 Comparison between arduino and processing language	

CHAPTER 1

INTRODUCTION

Motivation and Theoretical overview

The Internet of Things (IoT) is upon us. Sensors and embedded devices in automobiles, phones, watches, supermarkets, homes, roads and bridges, appliances and industrial and farm equipment, and wearable technology are already making new kinds of information available and changing the way information is produced, consumed, and experienced. IoT obviously represents a great opportunity for advances in information analysis. The connections between IoT and data storage and processing as well as machine learning are obvious and gaining attention already.

We will concentrate on the relationship between IoT and multiagent systems (MAS) research. We will introduce the technologies that enable IoT along with a range of representative applications. From these applications, we will abstract out the major challenges to realizing the vision of IoT. These include specification of computations; resource sharing; cooperation; enactment; engineering; and governance. Particular aspects of the IoT that are relevant to MAS include the heterogeneity of IoT components; power, connectivity, and computational resource constraints; asynchronous and delay-tolerant communication; multiple administrative domains with autonomous providers and consumers; multiple stakeholders with diverse and potentially conflicting requirements, including for resource usage and privacy.

The major objective of this project is to understand IoT applications as decentralized multiagent systems constructed of heterogeneous and autonomous entities. Doing so will enable them to systematically approach practical challenges that the IoT poses, such as coordination among devices and across organizations, and attendant concerns such as security and privacy, and to identify new research advances that will help address these challenges better.

Problem Statement

Traditional agriculture is characterized by unnecessary human interaction resulting in higher labor costs and susceptibility to severe weather events due to limited integration of data-driven decision support systems. The inability of smallholder farmers to invest in IoT technologies could be attributed to the prevailing uncertainty about the selling prices of agricultural produce due to variations in the market prices, lack of defined parameters about energy and water allocations, and energy use.

The tight production margins provide a minimal incentive for smallholder farmers to invest in new technologies such as IoT; this contrasts with large commercial producers who can easily acquire IoT systems. The comparative analysis of the two clearly indicates that cost was a critical impediment to the widespread use of IoT infrastructure .

Under an intense deployment of sensors, a greenhouse/farm is fitted with multiple interconnected sensor nodes across different points, which facilitate real-time monitoring of the areas of interest using WSNs. In contrast, a scattered configuration is characterized by the less-intense distribution of sensors to save costs.

Research objective

There is broad consensus among scholars that the optimization of IoT systems for greenhouses requires the careful selection of sensors, data acquisition, optimization, determination of the desired settings, and rule-based control .Considering that the conditions have to be optimized for different horticultural crops, there is a need for advanced techniques for data acquisition using Kalman filter prediction and related techniques that can forecast future environmental conditions using historical data.

In reality, the regulation of each parameter (temperature, moisture, pH, pesticides, humidity, UV radiation, rain, CO₂, and pressure) in smart greenhouses is a challenge especially in cases where there is inadequate historical data.

Presently, there is a wide array of IoT-based sensors for smart greenhouses, including plant growth sensors, temperature and humidity sensors, insect detection sensors, soil temperature, pH, and moisture sensors, and solar radiation, atmospheric pressure, wind speed, and CO₂ (and other gas) sensors, which rely on Bragg, piezoelectric, electrochemical, electromagnetic, and fiber-optic technologies for accurate assessment of the desired parameters. The parameters of interest include different wavelengths of light, photocurrent, fluorescence intensity, the fluorescent signal emitted by plant chlorophyll, optical density, and the electrochemical signal generated by enzyme-catalyzed redox reaction (SHA principle). Advances in research and design have resulted in the development of electromagnetic sensors for analyzing chlorophyll values and nitrogen concentration in plants; this approach relies on light reflectance and pulsating laser diodes. The technique was also proven useful in measuring the real-time plant physiology, including a plant's vegetative index, nutritional requirements, electrical conductivity, and magnetic susceptibility and conductivity (quad-phase).

The regulation and monitoring of ambient conditions in a greenhouse are critical considering that excess heat or humidity translates to large-scale plant damage. The ideal plant ambient temperature and relative humidity for optimal plant growth were 35 °C and 95%, respectively. Excess temperature and humidity are detrimental to optimal plant growth, given that they impair pollination, photosynthesis, leaf growth, and yield. The optimization of these parameters in smart greenhouses is partly impaired by the limited accuracy of available sensors. The accuracy of selected sensors is limited to 2–25%. Such low levels of accuracy could have disastrous consequences in the regulation of greenhouse microclimates, especially frost mitigation. In other cases, the accuracy of the IoT sensors is 99%. High levels of accuracy were achieved with smart stick sensors that provide live monitoring information about soil moisture and temperature. The information drawn from the smart stick sensors is synchronized with smart devices to obtain live feeds about variations in the physical parameters.

Scope of study

This includes

- Face recognition algorithms.
- Image processing using MATLAB.
- Use of MATLAB toolbox such as Image acquisition toolbox and computer vision toolbox. •

Accessing MS Excel spreadsheet in MATLAB using Spreadsheet Link EX

Research Methodology

Here we are trying to develop a system which automatically updates the data from the sensors in the thingspeak cloud. The system consists of a Arduino UNO setup along with a power supply along with a ESP8266 wifi module. The sensors include light, humidity, moisture and the actuators include exhaust fan, led light, water pump.

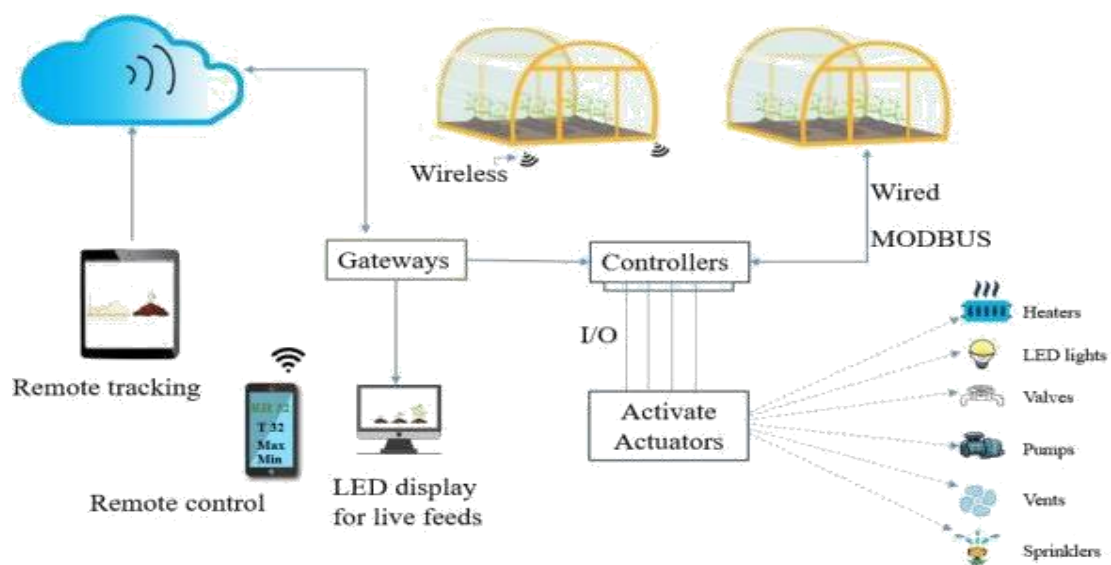


Fig 1.1 Model Diagram

CHAPTER 2

INTRODUCTION TO EMBEDDED SYSTEMS

INTRODUCTION TO EMBEDDED SYSTEM:

An embedded system can be defined as a computing device that does a specific focused job. Appliances such as the air-conditioner, VCD player, DVD player, printer, fax machine, mobile phone etc. are examples of embedded systems. Each of these appliances will have a processor and special hardware to meet the specific requirement of the application along with the embedded software that is executed by the processor for meeting that specific requirement. The embedded software is also called “firm ware”. The desktop/laptop computer is a general purpose computer. You can use it for a variety of applications such as playing games, word processing, accounting, software development and so on. In contrast, the software in the embedded systems is always fixed listed below:

Embedded systems do a very specific task; they cannot be programmed to do different things. . Embedded systems have very limited resources, particularly the memory. Generally, they do not have secondary storage devices such as the CDROM or the floppy disk. Embedded systems have to work against some deadlines. A specific job has to be completed within a specific time. In some embedded systems, called real-time systems, the deadlines are stringent. Missing a deadline may cause a catastrophe-loss of life or damage to property. Embedded systems are constrained for power. As many embedded systems operate through a battery, the power consumption has to be very low.

Some embedded systems have to operate in extreme environmental conditions such as very high temperatures and humidity.

Circuit Diagram

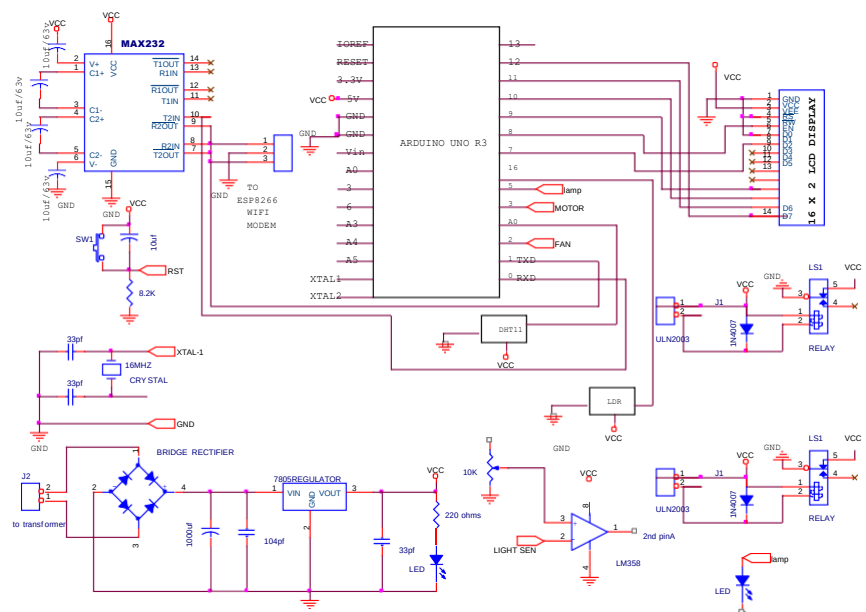


Fig.2.1Circuit diagram

Every embedded system consists of custom-built hardware built around a Central Processing Unit (CPU). This hardware also contains memory chips onto which the software is loaded. The software residing on the memory chip is also called the 'firmware'.

The operating system runs above the hardware, and the application software runs above the operating system. The same architecture is applicable to any computer including a desktop computer. However, there are significant differences. It is not compulsory to have an operating system in every embedded system. For small appliances such as remote control units, air conditioners, toys etc., there is no need for an operating system and you can write only the software specific to that application. For applications involving complex processing, it is advisable to have an operating system. In such a case, you need to integrate the application software with the operating system and then transfer the entire software on to the memory chip. Once the software is transferred to the memory chip, the software will continue to run for a long time you don't need to reload new software.

Now, let us see the details of the various building blocks of the hardware of an embedded system. As shown in Fig. the building blocks are;

- Central Processing Unit (CPU)
- Memory (Read-only Memory and Random Access Memory)
- Input Devices
- Central Processing Unit (CPU)
- Memory (Read-only Memory and Random Access Memory)
- Input Devices

CENTRAL PROCESSING UNIT(CPU):

The Central Processing Unit (processor, in short) can be any of the following: microcontroller, microprocessor or Digital Signal Processor (DSP). A micro-controller is a low-cost processor. Its main attraction is that on the chip itself, there will be many other components such as memory, serial communication interface, analog-to digital converter etc. So, for small applications, a micro-controller is the best choice as the number of external components required will be very less. On the other hand, microprocessors are more powerful, but you need to use many external components with them. DSP is

MEMORY:

The memory is categorized as Random Access Memory (RAM) and Read Only Memory (ROM). The contents of the RAM will be erased if power is switched off to the chip, whereas ROM retains the contents even if the power is switched off. So, the firmware is stored in the ROM. When power is switched on, the processor reads the ROM; the program is executed.

INPUT DEVICES:

Unlike the desktops, the input devices to an embedded system have very limited capability. There will be no keyboard or a mouse, and hence interacting with the embedded system is no easy task. Many embedded systems will have a small keypad-you press one key to give a specific command. A keypad may be used to input only the digits. Many embedded systems used in process control do not have any input device for user interaction; they take inputs from sensors or transducers that produce electrical signals that are in turn fed to other systems.

OUTPUT DEVICES:

The output devices of the embedded systems also have very limited capability. Some embedded systems will have a few Light Emitting Diodes (LEDs) to indicate the health status of the system modules, or for visual indication of alarms. A small Liquid Crystal Display (LCD) may also be used to display some important parameters.

COMMUNICATION INTERFACES:

When converting an RGB image to gray scale, consider the RGB values for each pixel and make as output a single value reflecting the brightness of that pixel. One of the approaches is to take the average of the contribution from each channel: $(R+G+B)/3$. However, since the perceived brightness is often dominated by the green component, a different, more “human oriented”, method is to consider a weighted average.

PERIPHERALS:

Peripherals are the various devices that are connected to the CPU, for performing various functions. Embedded systems talk with the outside world via peripherals, such as:

- ☐ Serial communication interfaces (SCI): RS-232, RS-422, RS-458 etc.
- ☐ Synchronous Serial communication interfaces (SSCI): I2C, JTAG, SPI, SSC and ESSI
- ☐ Universal Serial Bus (USB)
- ☐ Networks: Controller Area Network, etc.
- ☐ Timers: PLL(s), Capture/Compare and Time Processing units.
- ☐ Discrete I/O: General Purpose Input/output (GPIO).

Processors:

Processors are the key elements in any embedded system. They interact with the memory, where the various instructions of useful functions into a single IC package.

These functions are:

- ☐ The ability to execute a stored set of instructions to carry out user defined tasks.
- ☐ The ability to be able to access external memory chips to both read and writes data from and to the memory.

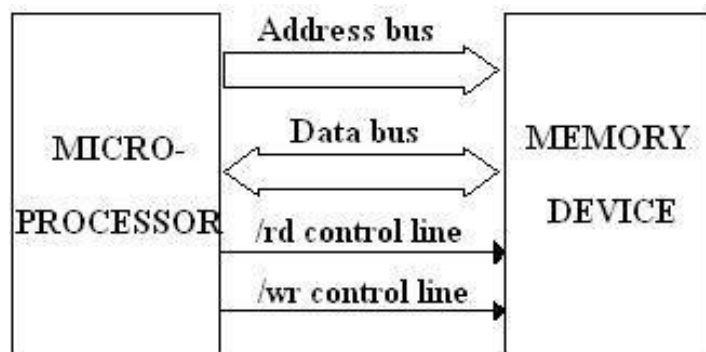


Fig-2.2: Communication between microprocessor and memory device

Reliability:

Embedded systems often reside in machines that are expected to run continuously for years without errors and in some cases recover by themselves if any error occurs. Therefore, the software is usually developed and tested more carefully than that for PC, and unreliable mechanical moving parts such as Disk drives, switches or buttons are avoided. Specific reliability issues may include:

- The system cannot safely be shut down for repair, or it is too inaccessible to repair. Solutions may involve subsystems with redundant spares that can be switched over to, or software “limp modes” that provide partial function. Examples include space systems, undersea cables, navigational beacons, bore-hole systems and automobiles.
- The system must be kept running for safety reasons. “Limp modes” are less tolerable. Often backups are selected by an operator. Examples include Aircraft, Navigation, Reactor control systems, safety-critical Chemical factory controls, Train signals and engines on single-engine Aircraft.
- The system will lose large amounts of money when shutdown: Telephone switches, Factory controls, Bridge and elevator controls, funds transfer and market making, automated sales and service.

Physically, embedded systems ranged from portable devices such as MP3 players, to large stationary installations like traffic lights, Factory controllers. In terms of complexity embedded systems can range from very simple with a single microcontroller chip to very complex with multiple units, peripherals and networks mounted inside a large chassis or enclosure.

CHAPTER 3

ANALYSIS OF PROJECT

OVERVIEW:

The Agriculture is the unquestionably the largest livelihood provider in India. With rising population, there is a need for increased agricultural production. In order to support greater production in farms, the requirement of the amount of fresh water used in irrigation also rises. Currently, agriculture accounts 83% of the total water consumption in India . Unplanned use of water inadvertently results in wastage of water. This suggests that there is an urgent need to develop systems that prevent water wastage without imposing pressure on farmers.

Over the past 15 years, farmers started using computers and software systems to organize their financial data and keep track of their transactions with third parties and also monitor their crops more effectively. In the Internet era, where information plays a key role in people's lives, agriculture is rapidly becoming a very data intensive industry where farmers need to collect and evaluate a huge amount of information from a diverse number of devices (eg., sensors, farming machinery etc.) in order to become more efficient in production and communicating appropriate information.

With the advent of open source Arduino boards along with cheap moisture sensors, it is viable to create devices that can monitor the soil moisture content and accordingly irrigating the fields or the landscape as a when needed. The proposed system makes use of microcontroller ATMEGA328P on Arduino Uno platform and IOT which enable farmers to remotely monitor the status of sprinklers installed on the farm by knowing the sensor values thereby, making the farmers' work much easier as thecan concentrate on other farm activities

BLOCK DIAGRAM:

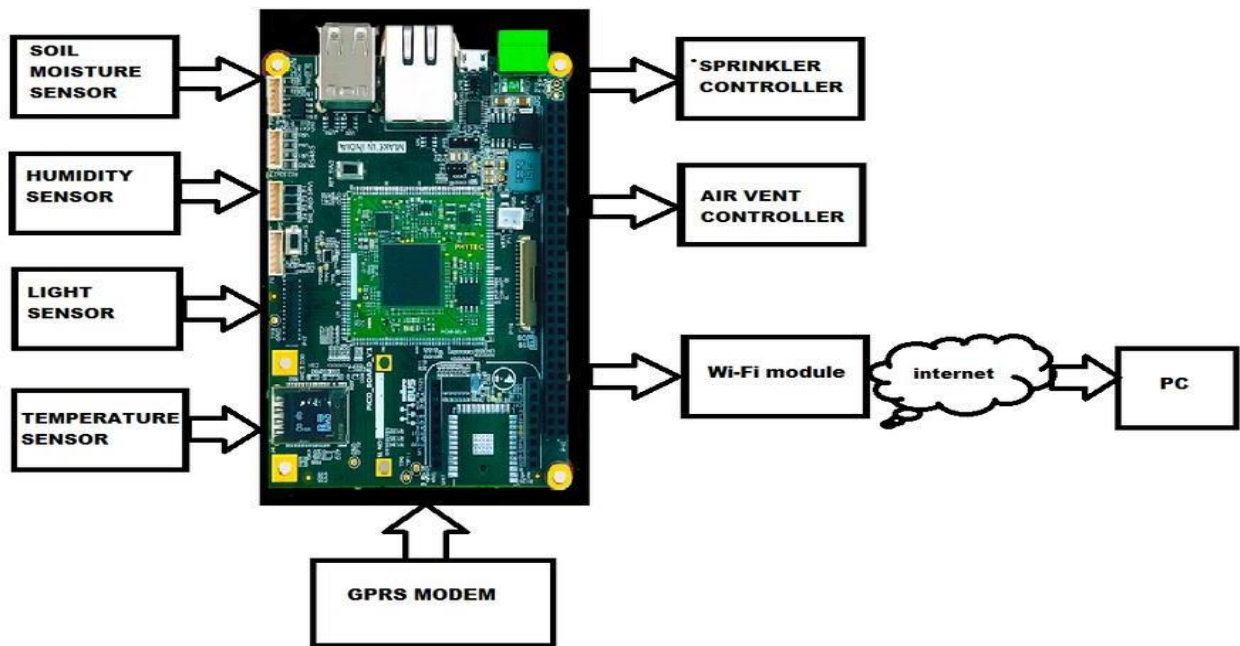


fig.3.1 Block Diagram

Circuit diagram:

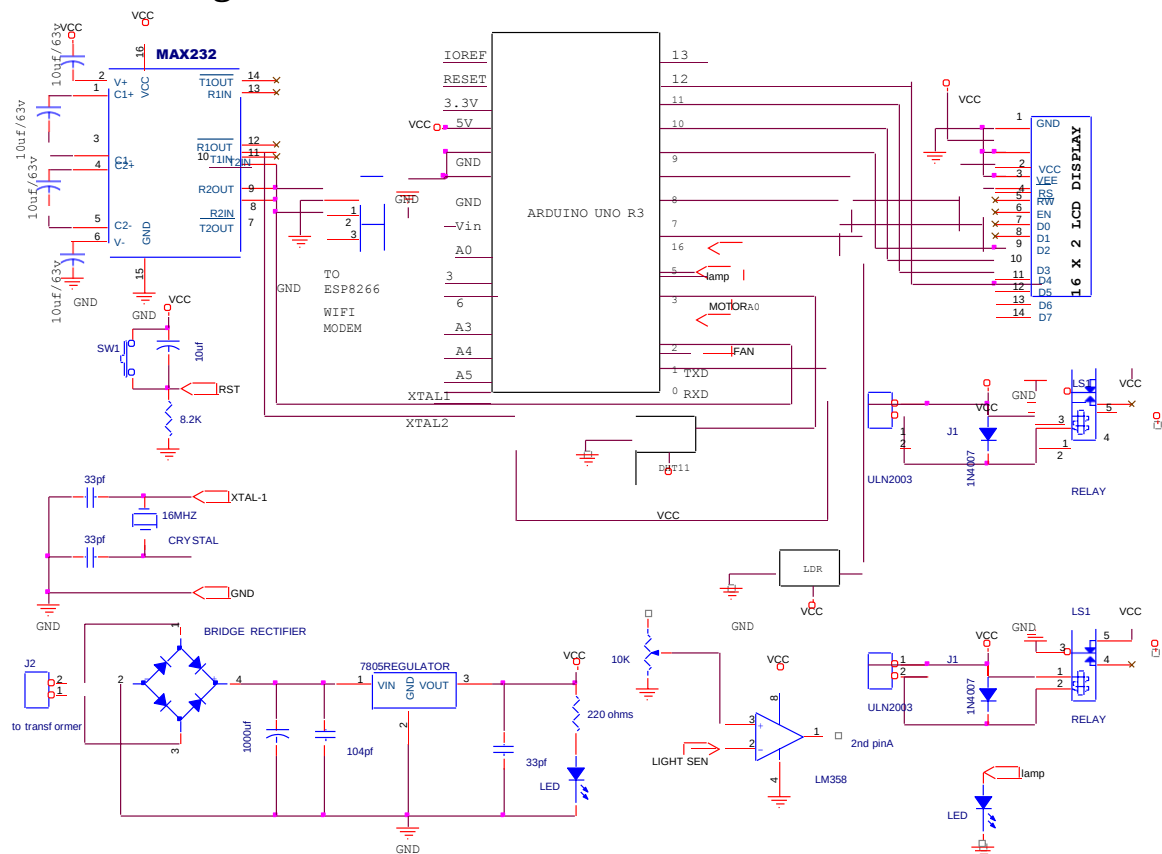


Fig.3.2 Circuit Diagram

STEPS OF WORKING:

- Switch on the power supply with a 12v step up transformer and make every component in active condition.
- LCD displays the values of values and initial displays as zero.
- Temperature sensor give output to ADC as input and this input is connected to Microcontroller.
- Speed of wind is proportional to the emf produced by the motor which is converted by ADC is collected by Microcontroller.
- The first comparator acts as reference and compares the level according to conditions which indirectly indicates the level/amount of rain water.
- The data from all the modules are collected in parallel to the microcontroller and according to the program it makes LCD to display the appropriate module data on screen.
- As Max232 is a serial data transfer the data from microcontroller to RS232 in the form of ASCII which is connected to laptop/ monitor screen which helps to look at the values of the modules same as on LCD.

DESCRIPTION OF THE CIRCUIT:

In the circuit description we will be discussing about how the interfacings of the modules are done and connections established between the microcontrollers. The heart of the circuit is the microcontroller and the program in it will be the key for functioning. Some of the important components used are Relays, LED's, Max 232, Motor and LCD.

The basic connections to the microcontroller are that pin 40 connected to Vcc, 20th pin for ground and pin 9 for Reset. Crystal oscillator is connected between 18 and 19th pins and it is the timer for the controller to work in a correct pattern. Without crystal oscillator the program code doesn't execute and there will be no synchronization between input and output signals. Here, port 0 pins (39-32) and port1 pins (2-8) are used to connect the LCD. Pins (21-28) which is port 2 of microcontroller is connected the keypad. Pin 17 are connected to motor which represents a fan and pins 14, 15 are connected for LEDS. Pin10and pin11 to a Max232 IC. Pin 9 is connection for reset circuit. Pin 12 is used for connecting PIR sensor and pin 28 for gas sensor.

Now coming to the description of individual modules and their importance. Firstly, there are two modes in the system one is automated mode and other is Manual mode. Initially the system is in auto mode by default are activated which takes action automatically and intimates the user through SMS. We can set Manual mode using the keypad connected to port2 which acts as input for key to change into other mode. In order to establish communication between GSM and microcontroller we used MAX232 Ic as voltage leveler and regulator. We can activate Auto mode using auto command through SMS.

CHAPTER 4

ARDUINO MICROCONTROLLER

Introduction:

Circumstances that we find ourselves in today in the field of microcontrollers had their beginnings in the development of technology of integrated circuits. This development has made it possible to store hundreds of thousands of transistors into one chip. That was a prerequisite for production of microprocessors, and the first computers were made by adding external peripherals such as memory, input-output lines, timers and other. Further increasing of the volume of the package resulted in creation of integrated circuits. These integrated circuits contained both processor and peripherals. That is how the first chip containing a microcomputer, or what would later be known as a microcontroller came about.

Definition of a Microcontroller :

Microcontroller, as the name suggests, are small controllers. They are like single chip computers that are often embedded into other systems to function as processing/controlling unit. For

example, the remote control you are using probably has microcontrollers inside that do decoding and other controlling functions. They are also used in automobiles, washing machines, microwave ovens, toys ... etc., where automation is needed.

The key features of microcontrollers include:



Integration of Functionality



Microcontrollers sometimes are called single-chip computers because they have on-chip memory and I/O circuitry and other circuitries that enable them to function as small standalone computers without other supporting circuitry.



Field Programmability, Flexibility



Microcontrollers often use EEPROM or EPROM as their storage device to allow field programmability so they are flexible to use. Once the program is tested to be correct then large quantities of microcontrollers can be programmed to be used in embedded systems.



Easy to Use

Assembly language is often used in microcontrollers and since they usually follow RISC architecture, the instruction set is small. The development package of microcontrollers often

Includes an assembler, a simulator, a programmer to "burn" the chip and a demonstration board. Some packages include a high level language compiler such as a C compiler and more sophisticated libraries.

Most microcontrollers will also combine other devices such as:

- A Timer module to allow the microcontroller to perform tasks for certain time periods.
- A serial I/O port to allow data to flow between the microcontroller and other devices such as a PC or another microcontroller.
- An ADC to allow the microcontroller to accept analogue input data for processing

•

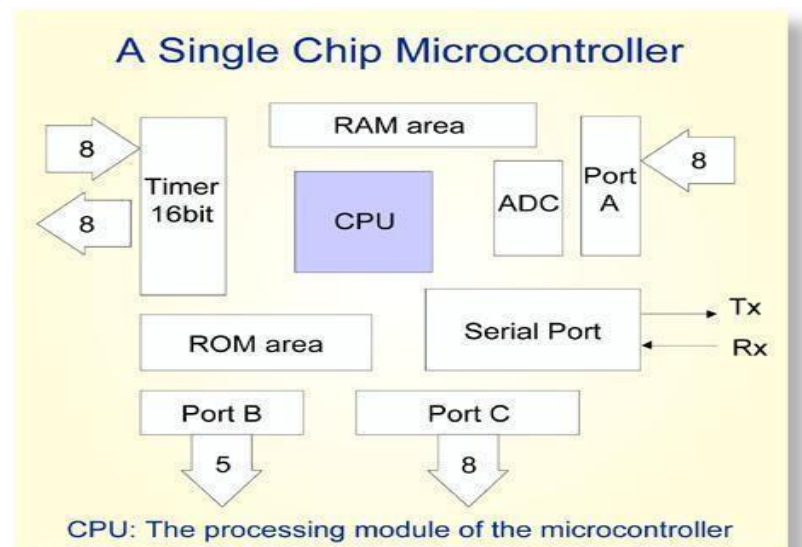


Fig 4.1: Showing a typical microcontroller device and its different subunits

Microcontrollers versus Microprocessors:

Microcontroller differs from a microprocessor in many ways. First and the most important is its functionality. In order for a microprocessor to be used, other components such as memory, or components for receiving and sending data must be added to it. In short that means that microprocessor is the very heart of the computer. On the other hand, microcontroller is designed to be all of that in one. No other external components are needed for its application because all necessary peripherals are already built into it. Thus, we save the time and space needed to construct devices.

Features of ATmega328:

Microcontroller can be described as a computer embedded on a rather small circuit board. To describe the function of a microcontroller more precisely, it is a single chip that can perform various calculations and tasks, and send/receive signals from other devices via the available pins. Precisely what tasks and communication with the world it does, is what is governed by what instructions we give to the Microcontroller. It is this job of telling the chip what to do, is what we refer to as programming on it.

However, the uC by itself cannot accomplish much; it needs several external inputs: power, for one; a steady clock signal, for another. Also, the job of programming it has to be accomplished by an external circuit. So typically, a uC is used along with a circuit which provides these things to it; this combination is called a microcontroller board. The Arduino Uno that you have received is one such microcontroller board. The actual microcontroller at its heart is the chip called **Atmega328**. The advantages that Arduino offers over other microcontroller boards are largely in terms of reliability of the circuit hardware as well as the ease of programming and using it.

ARDUINO:

Arduino interface boards provide the engineers, artists, designers, hobbyists and anyone who tinker with technology with a low-cost, easy-to-use technology to create their creative, interactive objects, useful projects etc., A whole new breed of projects can now be built that can be controlled from a computer.

WHAT IS ARDUINO?



Figure 4.2 ARDUINO UNO

Arduino is an open source electronics prototyping platform based on flexible, easy-to-use hardware and software. It's intended for artists, designers, hobbyists, and anyone interested in creating interactive objects or environments. It's an open-source physical computing platform based on a microcontroller board, and a development environment for writing software for the board.

In simple words, Arduino is a small microcontroller board with a USB plug to connect to your computer and a number of connection sockets that can be wired up to external electronics, such as motors, relays, light sensors, laser diodes, loudspeakers, microphones, etc., They can either be powered through the USB connection from the computer or from a 9V battery. They can be controlled from the computer or programmed by the computer and then disconnected and allowed to work independently.

Anyone can buy this device through online auction site or search engine. Since the Arduino is an open-source hardware designs and create their own clones of the Arduino and sell them, so the market for the boards is competitive. An official Arduino costs about \$30, and a clone often less than \$20.

The name "Arduino" is reserved by the original makers. However, clone Arduino designs often have the letters "duino" on the end of their name, for example, Fredonia or DFRduino. The software for programming your Arduino is easy to use and also freely available for Windows, Mac, and LINUX computers at no cost.

HISTORY OF ARDUINO:

While teaching a physical computing class at the Interaction Design Institute Ivrea in 2005, Massimo Benzie's students were unwilling to spend the 76 euros for the BASIC Stamp microcontrollers commonly used in such applications. Benzie and his colleagues looked for alternatives, finally settling on the *wiring* platform developed by one of Benzie's students. In his own words:

Design Goals

- ☐ Work with a Mac (as most design students use one)
- ☐ USB connectivity (MacBook's don't have serial ports)
- Cheap(About 20 euros, the cost of pizza in Europe.

More powerful than a basic stamp

- ☐ Looks nice
- ☐ Something you could build /fix yourself

PLATFORM

HARDWARE

ARDUINO Board Layout

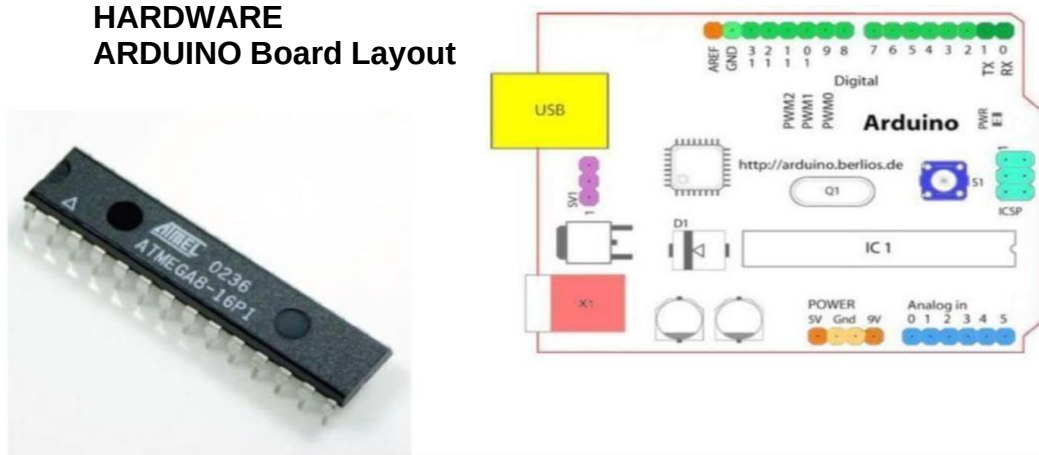


Figure 4.3 ARDUINO board layout

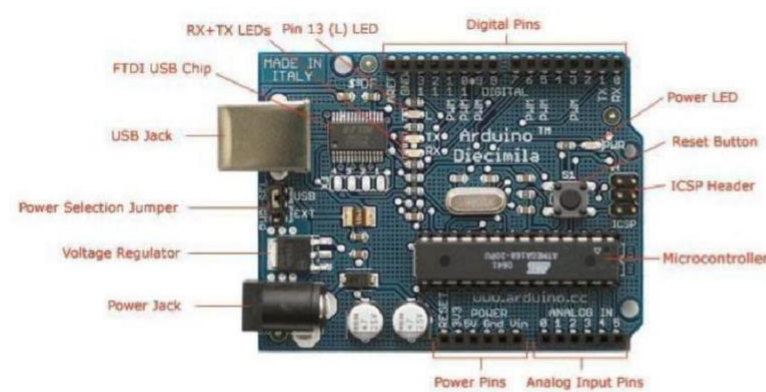


Figure 4.4 ARDUINO pin diagram



Fig4.5: Ac adapter

- 16 MHZ
- ☐ 8 Kbyte Flash RAM (1K taken by the boot

loader)

- ☐ 1 Kbyte RAM (eg. for auto/local variables and stack)
- ☐ 14 digital Input/output Ports

Single chip USB to async. Serial data transfer interface

- ☐ USB 2.0 compatible
- ☐ Transmit and receive LED five signals
- ☐ 256 Byte receive, 128 Byte transmit buffer
- ☐ Data transfer rate from 300bits/sec to 2 Mb/sec

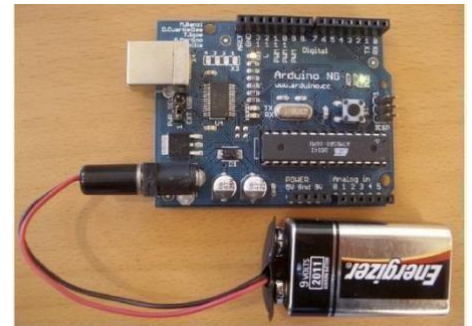


Fig4.6 Battery

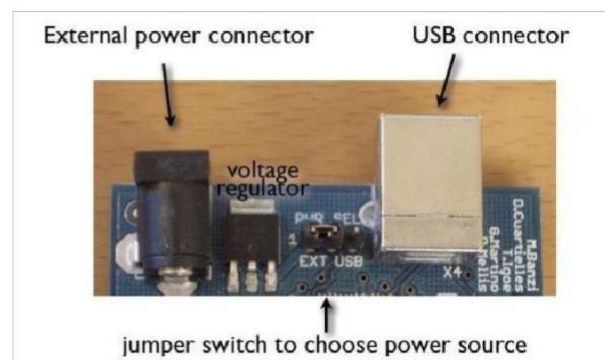


Figure 4.7 arduino can run off with USB or External power supply

The power requirement for ARDUINO is 9 to 12V DC, 250mA or more, 2.1mm plug, Centre pin positive.

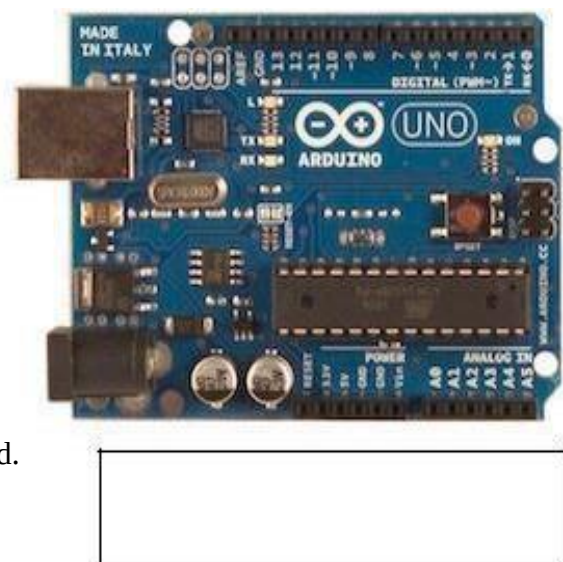
ARDUINO flavors!!

There have been many revisions of the USB Arduino.

Some of them are

1. Arduino UNO:

This is the latest revision of the basic Arduino USB. Board. It connects to the computer with a standard. USB cable and contains everything else you need to program and use the board. It can be extended with a variety of



shields: custom daughter-boards with specific features.
It is similar to the Duemilanove,

But has a different USB-to-serial chip the ATmega8U2, and newly designed labeling to make inputs and outputs easier to identify.

2. Arduino Mega 2560:

A larger, more powerful Arduino board. Has digital pins, PWM pins, analog inputs, Serial ports, etc. The version of the Mega released With The Uno, this version features with the Atmega2560 , which had twice the memory And uses Atmega 8U2 for USB to serial communication .

Fig 4.8: Arduino Uno



Fig 4.9: Arduino Mega 256

Arduino	Processor	Flash KiB	EEPROM KiB	SRAM KiB	Digital I/O pins	...with PWM	Analog input pins	USB Interface type	Dimensions inches	Dimensions mm
Diecimila	ATmega168	16	0.5	1	14	6	6	FTDI	2.7" x 2.1"	68.6mm x 53.3mm
Duemilanove	ATmega168/328P	16/32	0.5/1	1/2	14	6	6	FTDI	2.7" x 2.1"	68.6mm x 53.3mm
Uno	ATmega328P	32	1	2	14	6	6	ATmega8U2	2.7" x 2.1"	68.6mm x 53.3mm
Mega	ATmega1280	128	4	8	54	14	16	FTDI	4" x 2.1"	101.6mm x 53.3mm
Mega2560	ATmega2560	256	4	8	54	14	16	ATmega8U2	4" x 2.1"	101.6mm x 53.3mm
Fio	ATmega328P	32	1	2	14	6	8	None	1.6" x 1.1"	40.6mm x 27.9mm
Nano	ATmega168 or ATmega328	16/32	0.5/1	1/2	14	6	8	FTDI	1.70" x 0.73"	43mm x 18mm
LilyPad	ATmega168V or ATmega328V	16	0.5	1	14	6	6	None	2" ø	50mm ø

Table 4.1 Different Flavors of ARDUINO with their Configuration

Basic Terminologies in

ARDUINO: 1. Analog to digital

converter (ADC)

The process of Analog to digital conversion is shown in figure. The Arduino has 10 bits of Resolution when reading analog signals

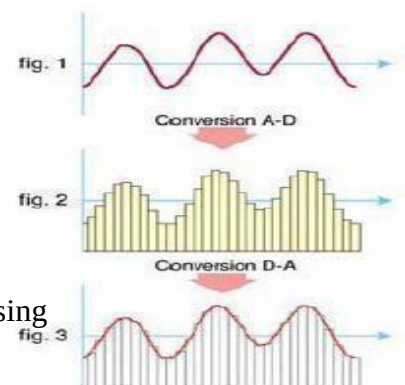
$$2^{10} = 1024 \text{ increments}$$

Influence also by how fast you sample

Pulse width modulation (PWM)

The Arduino has 8bit of resolution, when outputting a signal using PWM. The range of output voltage is from 0 to 5 Volts $2^8 = 255$ Increments

Average of on/off(digital signals to make an average voltage), Duty cycle in 100% of 5Volts.



Why Arduino is popular:

Here are five reasons why the Arduino is more popular than beagle board:

- **Starter Projects:** Editing and rewriting is often easier than writing from scratch. It's the same with electronics. It's easier to mod an idea than start with a blank slate. That's where the Beagle Board falls short. "It has virtually no example application that you can just copy and hack to learn from," says Massimo Benzie, one of the co-founders of the Arduino project. It's a chicken-and-egg problem for the Beagle Board. Unless there are more example codes out there, it is difficult to draw in the audience. And without the audience it is challenging to get enough sample projects into the community.
- **Cost and Durability:** At \$30 apiece, an Arduino is an inexpensive investment for someone who wants to try it out. "It's the price of a few sandwiches," says Torrone. Compare that to the BeagleBoard-xM, which costs \$180. One reason why the Arduino is so cheap is because it is easy to clone. The microcontroller is completely open source so the "components are all

commodity,” says Torrone. With the Beagle Board, hobbyists don’t have the same amount of freedom. They have to work closely with

Texas Instruments or its partners, says Torrone. Arduino is also very resilient. Drop it, smash it and it still stays alive. Add to that its low-power requirement, and the product becomes a must-have for DIYers. An Arduino can run on a 9V-battery for days. “The Beagle Board is fast and powerful but that also means lots of energy is needed, which makes it difficult for simple projects,” says Torrone

A Thriving Community: Arduino’s popularity means it’s easy to get started. Companies such as Adafruit, Spark Fun and Liquid ware not only sell chips, but they also host blogs that suggest ideas on how to use your Arduino while providing extensive project plans to guide you in completing your creations. Will Chellman, a student who has played with Arduino for years, says he’s now experimenting with the Beagle Board. But finding documentation and information to work off is not easy, he says. The lack of well-documented projects done with the BeagleBoard can be intimidating to new users as well, says Benzie. “There’s lots of interesting stuff (about the BeagleBoard) but it is very technical,” he wrote in a comment recently on Gadget Lab in response to the launch of BeagleBoard-xM. Benzie says BeagleBoard documentation is also scattered and fragmented. “Parts of it have aged and you spend quite a bit of time jumping from wikis to mailing list to track which specific bit of documentation applies to your board, boot loader etc.,” he says.

- **Maturity is the key:** Arduino has had a head start on the BeagleBoard. By October 2008, about 50,000 Arduino boards had already been shipped. That year, the first Beagle Boards started making their way into the hands of hardware enthusiasts. The BeagleBoard is just two years old. Since it hasn’t been around long enough, there’s not enough people building apps based on it,” says Chellman. That’s not to say that BeagleBoard isn’t catching up. Earlier this month, we showed five projects ranging from a video wall to the iPad of ham radios that use the BeagleBoard. There’s also a build-your-own tablet kit that is based off the BeagleBoard.
- **Simple is attractive:** With its single-board computer configuration, 1-GHz processing power and the choice of accessories, the BeagleBoard is a creative engineer’s dream come true. But the same reasons make it intimidating to those who want to geek out on a DIY project but don’t have the technical know-how. Arduino users point out that it is simple to connect external sensors to the board, and the example codes out there make it easy to get started quickly. Arduino is a simple system designed for creative people with little or “no prior knowledge of electronics,” says Banzi. “It’s cheap and open source with lots of

documentation written in a not too technical language. Above all, it has a very welcoming attitude towards beginners and tries not to scare them too much.”

Conclusion and future scope

Over the years, Arduino has gone out to become a huge success and a common name among students. With Google deploying it, people’s imagination has gone out too much higher level than before. A developer in the annual GOOGLE IO conference said “when Arduino and Android coming together, this really proves “INFINITY EXISTS” in the future”. I think a study on Arduino and practical experiments on Arduino must be added for UG courses of engineering, to help students to leverage their talents, and imagination.

CHAPTER 5

HARDWARE MODULES (Gsm and ESP8266)

5.1 RS232:

fig.5.1RS232

The USB to RS232 serial communications interface provides RS232 serial communication interfaces to computers with USB support. RS-232 full duplex serial port furnished with a 9 pin D-type male connector similar to which can be found on back of PC. The port is fully configured with two data, six control I/O lines and one signal return. Driver provided for Win98, ME, 2K, XP, Vista, Mac and Linux.



USB to RS232 Converter is an intelligent module which connects to a PC Universal Serial Bus port providing a high-speed asynchronous RS-232 serial port. The USB-232 Converter provides easy connectivity between the PC and standard Communication ports, not requiring PC reconfiguration.

The USB to RS232 module is a cost-effective way to convert RS232 interface to a USB interface. When connected to a PC USB port the USB to RS232 module is automatically detected and is installed as a native COM port which is compatible with any existing serial communication application. Multiple modules can be installed when using USB hubs thus allowing a hassle-free configuration of a multi serial system without any IRQ or DMA.

Specifications:

Parameter	Value
USB Driver Interface	Works with existing COM Port PC applications (Installs Virtual COM port on PC to which existing software can connect)
Communication speed	75 to 6MBPS baud
Power consumption	50mA @ 5 volts (power provided by the PC's USB port.)
Temperature Range	0°C to 50°C.

Modes of Operation	RS-232
USB Compliant	USB Specification 2.0 compliant; full-speed (12 Mbps)
Asynchronous Serial Data BUS (UART)	All handshaking and modem interface signals

Data formats supported:	- Data bits: 5, 6, 7, and 8 - Stop bits: 1, 1.5, and 2
Buffer	256 Byte receive buffer; 256 byte transmit buffer
Handshaking	Hardware or X-On/X-Off handshaking supported

Table 5.1: Rs232 specifications

Asynchronous Serial Data Bus (UART) Interface:

The UART interface consists of the TX (transmit) and RX (receive) data signals as well as the RTS, CTS, DSR, DTR, DCD, and RI control signals. The UART supports RTS/CTS, DSR/DTR, and X-On/X-Off handshaking. The UART is programmable to support a variety of data formats and baud rates. The data format and baud rate programmed into the UART is set during COM port configuration on the PC. The data formats and baud rates available are listed in Table below.

Parameter	Value
Data Bits	5, 6, 7, and 8
Stop Bits	1, 1.5, and 2
Parity Type	None, Even, Odd, Mark, Space
Baud Rates	75 bps to 6 MBPS

Table 5.2: UART parameters

System Requirements

- A PC with a minimum of a 75MHz Pentium, or equivalent
- A minimum of 16M bytes of RAM
- One available USB type port compliant with USB1.1
- Drivers are included on our software CD
- Supported Operating Systems: Windows 2000/XP/Server 2003/Vista (v5.0), Macintosh OSX, Macintosh 9, Linux 2.4, Linux 2.6.

Features

- Computer interface: USB V1.1 or USB 2 Plug and Play.
- Operational system virtual serial port driver.
Supports Windows 98/ME/XP/2000/CE, MAC & Linux 2.4.20 or superior.
- Interface: RS232
- Transmission rate: from 75 bps to 6Mbps.

- Maximum RS232 cable length: 5 m
- Power: from the USB port.
- USB connection: B type connector
- A 1.5 m cable with plugs B and A is provided with the module.
- RS232 male connector: Accepts female type serial connections.
- ABS enclosure

Connection to the System

The USB-RS232 Converter is equipped with a USB cable and a USB type A connector for connection to a PC USB port and a 9 pin D Type Connector for Serial Port. Unit's 9 pin D Type Male Connector (Similar to COM port found on back of PC).

GSM Module:

What is GSM?

GSM (Global System for Mobile communications) is an open, digital cellular technology used for transmitting mobile voice and data services.

GSM (Global System for Mobile communication) is a digital mobile telephone system that is widely used in Europe and other parts of the world. GSM uses a variation of Time Division Multiple Access (TDMA) and is the most widely used of the three digital wireless telephone technologies (TDMA, GSM, and CDMA).

GSM was first introduced in 1991. As of the end of 1997, GSM service was available in more than 100 countries and has become the *de facto* standard in Europe and Asia.

What does GSM offer?

GSM supports voice calls and data transfer speeds of up to 9.6 kbit/s, together with the transmission of SMS (Short Message Service).

GSM operates in the 900MHz and 1.8GHz bands in Europe and the 1.9GHz and 850MHz bands in the US. The 850MHz band is also used for GSM and 3G in Australia, Canada and many South American countries. By having harmonized spectrum across most of the globe, GSM's international roaming capability allows users to access the same services when travelling abroad as at home. This gives consumers seamless and same number connectivity in more than 218 countries.

Terrestrial GSM networks now cover more than 80% of the world's population. GSM satellite roaming has also extended service access to areas where terrestrial coverage is not available.

History

In 1982, the European Conference of Postal and Telecommunications Administrations (CEPT) created the Group Special Mobile (GSM) to develop a standard for a mobile telephone system that could be used across Europe. In 1987, a memorandum of understanding was signed by 13 countries to develop a common cellular telephone system across Europe. Finally the system created by SINTEF lead by Torleiv Maseng was selected.

In 1989, GSM responsibility was transferred to the European Telecommunications Standards Institute (ETSI) and phase I of the GSM specifications were published in 1990. The first GSM network was launched in 1991 by Radiolinja in Finland with joint technical infrastructure maintenance from Ericsson. By the end of 1993, over a million subscribers were using GSM phone networks being operated by 70 carriers across 48 countries.

GSM Frequencies

GSM networks operate in a number of different frequency ranges (separated into GSM frequency ranges for 2G and UMTS frequency bands for 3G). Most 2G GSM networks operate in the 900 MHz or 1800 MHz bands. Some countries in the Americas (including Canada and the United States) use the 850 MHz and 1900 MHz bands because the 900 and 1800 MHz frequency bands were already allocated.

GSM-900 uses 890–915 MHz to send information from the mobile station to the base station (uplink) and 935–960 MHz for the other direction (downlink), providing 124 RF channels (channel numbers 1 to 124) spaced at 200 kHz. Duplex spacing of 45 MHz is used. In some countries the GSM-900 band has been extended to cover a larger frequency range. There eight radio timeslots (giving eight burst periods) grouped into what is called a TDMA frame. Half rate channels use alternate frames in the same timeslot. The transmission power in the handset is limited to a maximum of 2 watts in GSM850/900 and 1 watt in GSM1800/1900.

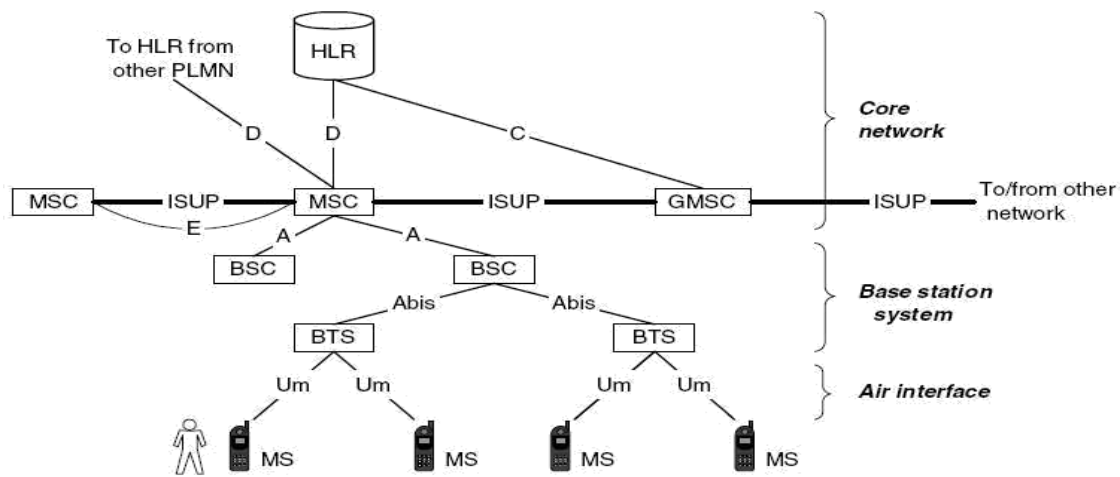


Figure 1.1 GSM network architecture

Fig.5.2 GSM network Architecture

5.3 ESP8266

Introduction

In 2014, an ESP8266 Wi-Fi module was introduced and developed by third-party manufacturers like AI thinkers, which is mainly utilized for IoT-based embedded applications development. It is capable of handling various functions of the Wi-Fi network from another application processor.

It is a SOC (System On-chip) integrated with a TCP/IP protocol stack, which can provide microcontroller access to any type of Wi-Fi network. This article deals with the pin configuration, specifications, circuit diagram, applications, and alternatives of the ESP8266 Wi-Fi module.

What is Esp8266?

An ESP8266 Wi-Fi module is a SOC microchip mainly used for the development of end-point IoT (Internet of things) applications. It is referred to as a standalone wireless transceiver, available at a very low price. It is used to enable the internet connection to various applications of embedded systems.

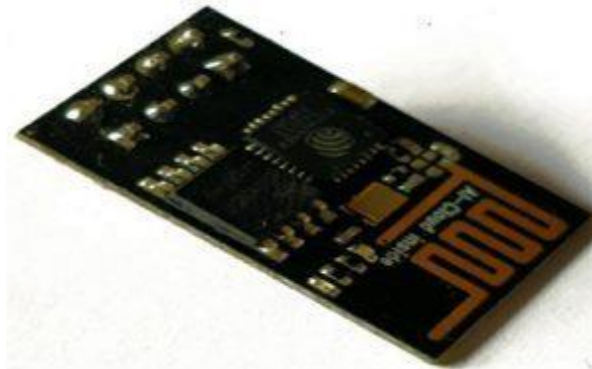


fig.5.3ESP8266

Espressif systems designed the ESP8266 Wi-Fi module to support both the TCP/IP capability and the microcontroller access to any Wi-Fi network. It provides the solutions to meet the requirements of industries of IoT such as cost, power, performance, and design.

It can work as either a slave or a standalone application. If the ESP8266 Wi-Fi runs as a slave to a microcontroller host, then it can be used as a Wi-Fi adaptor to any type of microcontroller using UART or SPI. If the module is used as a standalone application, then it provides the functions of the microcontroller and Wi-Fi network.

The ESP8266 Wi-Fi module is highly integrated with RF balun, power modules, RF transmitter and receiver, analog transmitter and receiver, amplifiers, filters, digital baseband, power modules,

external circuitry, and other necessary components. The ESP8266 Wi-Fi module is a microchip shown in the figure below.

A set of AT commands are needed by the microcontroller to communicate with the ESP8266 Wi-Fi module. Hence it is developed with AT commands software to allow the Arduino Wi-Fi functionalities, and also allows loading various software to design the own application on the memory and processor of the module.

The processor of this module is based on the Tensilica Xtensa Diamond Standard 106 micro and operates easily at 80 MHz. There are different types of ESP modules designed by third-party manufacturers. They are,

- ESP8266-01 designed with 8 pins (GPIO pins -2)
- ESP8266-02 designed with 8 pins (GPIO pins -3)
- ESP8266-03 designed with 14 pins (GPIO pins- 7)
- ESP8266-04 designed with 14 pins (GPIO pins- 7)

The ESP8266 Wi-Fi module comes with a boot ROM of 64 KB, user data RAM of 80 KB, and instruction RAM of 32 KB. It can support 802.11 b/g/n Wi-Fi network at 2.4 GHz along with the features of I2C, SPI, I2C interfacing with DMA, and 10-bit ADC. Interfacing this module with the microcontroller can be done easily through a serial port. An external voltage converter is required only if the operating voltage exceeds 3.6 Volts. It is most widely used in robotics and IoT applications due to its low cost and compact size.

Pin Configuration/Pin Diagram

The **ESP8266 Wi-Fi module pin configuration/pin diagram** is shown in the figure below. The ESP8266-01 Wi-Fi module runs in two modes. They are;

Fig.5.4 Pin Configuration of ESP8266

Flash Mode: When GPIO-0 and GPIO-1 pins are active high, then the module runs the program, which is uploaded into it.

UART Mode: When the GPIO-0 is active low and GPIO-1 is active high, then the module works in programming mode with the help of either serial communication or Arduino board.

ESP8266 Wi-Fi Module Specifications

The **ESP8266 Wi-Fi module specifications or features** are given below.

- It is a powerful Wi-Fi module available in a compact size at a very low price.
- It is based on the L106 RISC 32-bit microprocessor core and runs at 80 MHz
- It requires only 3.3 Volts power supply
- The current consumption is 100 m Amps
- The maximum Input/Output (I/O) voltage is 3.6 Volts.
- It consumes 100 mA current
- The maximum Input/Output source current is 12 mA
- The frequency of built-in low power 32-bit MCU is 80 MHz
- The size of flash memory is 513 kb
- It is used as either an access point or station or both
- It supports less than 10 microAmps deep sleep
- It supports serial communication to be compatible with several developmental platforms such as Arduino

- It is programmed using either AT commands, Arduino IDE, or Lua script
- It is a 2.4 GHz Wi-Fi module and supports WPA/WPA2, WEP authentication, and open networks.
- It uses two serial communication protocols like I2C (Inter-Integrated Circuit) and SPI (Serial Peripheral Interface).
- It provides 10-bit analog to digital conversion
- The type of modulation is PWM (Pulse Width Modulation)
- UART is enabled on dedicated pins and for only transmission, it can be enabled on GPIO2.
- It is an IEEE 802.11 b/g/n Wi-Fi module with LNA, power amplifier, balun, integrated TR switch, and matching networks.
- GPIO pins – 17
- Memory Size of instruction RAM – 32 KB
- The memory size of instruction cache RAM – 32 KB
- Size of User-data RAM- 80 KB
- Size of ETS systems-data RAM – 16 KB

5.4 Introduction to AT Commands :

AT commands are instructions used to control a modem. AT is the abbreviation of Attention. Every command line starts with "AT" or "at". That's why modem commands are called AT commands. Many of the commands that are used to control wired dial-up modems, such as ATD (Dial), ATA (Answer), ATH (Hook control) and ATO (Return to online data state), are also supported by GSM/GPRS modems and mobile phones. Besides this common AT command set, GSM/GPRS modems and mobile phones support an AT command set that is specific to the GSM technology, which includes SMS-related commands like AT+CMGS (Send SMS message), AT+CMSS (Send SMS message from storage), AT+CMGL (List SMS messages) and AT+CMGR (Read SMS messages).

Note that the starting "AT" is the prefix that informs the modem about the start of a command line. It is not part of the AT command name. For example, D is the actual AT command name in ATD and +CMGS is the actual AT command name in AT+CMGS. However, some books and web sites use them interchangeably as the name of an AT command.

Here are some of the tasks that can be done using AT commands with a GSM/GPRS modem or mobile phone:

- Get basic information about the mobile phone or GSM/GPRS modem. For example, name of manufacturer (AT+CGMI), model number (AT+CGMM), IMEI number (International Mobile Equipment Identity) (AT+CGSN) and software version (AT+CGMR).
- Get basic information about the subscriber. For example, MSISDN (AT+CNUM) and IMSI number (International Mobile Subscriber Identity) (AT+CIMI).

- ❑ Get the current status of the mobile phone or GSM/GPRS modem. For example, mobile phone activity status (AT+CPAS), mobile network registration status (AT+CREG), radio signal strength (AT+CSQ), battery charge level and battery charging status (AT+CBC).
- ❑ Establish a data connection or voice connection to a remote modem (ATD, ATA, etc).
- ❑ Send and receive fax (ATD, ATA, AT+F*).
- ❑ Send (AT+CMGS, AT+CMSS), read (AT+CMGR, AT+CMGL), write (AT+CMGW) or delete (AT+CMGD) SMS messages and obtain notifications of newly received SMS messages (AT+CNMI).
- ❑ Read (AT+CPBR), write (AT+CPBW) or search (AT+CPBF) phonebook entries.
- ❑ Perform security-related tasks, such as opening or closing facility locks (AT+CLCK), checking whether a facility is locked (AT+CLCK) and changing passwords (AT+CPWD).
(Facility lock examples: SIM lock [a password must be given to the SIM card every time the mobile phone is switched on] and PH-SIM lock [a certain SIM card is associated with the mobile phone. To use other SIM cards with the mobile phone, a password must be entered.])
- ❑ Control the presentation of result codes / error messages of AT commands. For example, you can control whether to enable certain error messages (AT+CMEE) and whether error messages should be displayed in numeric format or verbose format (AT+CMEE=1 or AT+CMEE=2).
- ❑ Get or change the configurations of the mobile phone or GSM/GPRS modem. For example, change the GSM network (AT+COPS), bearer service type (AT+CBST), radio link protocol parameters (AT+CRLP), SMS center address (AT+CSCA) and storage of SMS messages (AT+CPMS).
- ❑ Save and restore configurations of the mobile phone or GSM/GPRS modem. For example, save (AT+CSAS) and restore (AT+CRES) settings related to SMS messaging such as the SMS center address.

Basic Commands and Extended Commands

There are two types of AT commands: basic commands and extended commands.

Basic commands are AT commands that do not start with "+". For example, D (Dial), A (Answer), H (Hook control) and O (Return to online data state) are basic commands.

Extended commands are AT commands that start with "+". All GSM AT commands are extended commands. For example, +CMGS (Send SMS message), +CMSS (Send SMS message from storage), +CMGL (List SMS messages) and +CMGR (Read SMS messages) are extended commands.

General syntax of AT Commands:

The general syntax of extended AT commands is straightforward. The syntax rules are provided below. The syntax of basic AT commands is slightly different. We will not cover the Syntax of basic AT commands in this SMS tutorial since all SMS messaging commands are extended AT commands.

Syntax rule 1. All command lines must start with "AT" and end with a carriage return character. (We will use <CR> to represent a carriage return character in this SMS tutorial.) In a terminal program like HyperTerminal of Microsoft Windows, you can press the Enter key on the keyboard to output a carriage return character.

Example: To list all unread inbound SMS messages stored in the message storage area, type "AT", then the extended AT command "+CMGL", and finally a carriage return character, like this:

AT+CMGL<CR>

Syntax rule 2. A command line can contain more than one AT command. Only the first AT command should be prefixed with "AT". AT commands in the same command-line string should be separated with semicolons.

Example: To list all unread inbound SMS messages stored in the message storage area and obtain the manufacturer name of the mobile device, type "AT", then the extended AT command "+CMGL", followed by a semicolon and the next extended AT command "+CGMI": AT+CMGL; +CGMI<CR>

An error will occur if both AT commands are prefixed with "AT", like this:

AT+CMGL; AT+CGMI<CR>

Syntax rule 3. A string is enclosed between double quotes.

Example: To read all SMS messages from message storage in SMS text mode (at this time you do not need to know what SMS text mode is. More information will be provided later in this SMS tutorial), you need to assign the string "ALL" to the extended AT command +CMGL, like this:

AT+CMGL="ALL"<CR>

Syntax rule 4. Information responses and result codes (including both final result codes and unsolicited result codes) always start end with a carriage return character and a linefeed character.

Example: After sending the command line "AT+CGMI<CR>" to the mobile device, the mobile device should return a response similar to this:

<CR><LF>Nokia<CR><LF>
<CR><LF>OK<CR><LF>

The first line is the information response of the AT command +CGMI and the second line is the final result code. <CR> and <LF> represent a carriage return character and a linefeed character respectively. The final result code "OK" marks the end of the response. It indicates no more data will be sent from the mobile device to the computer / PC.

When a terminal program such as HyperTerminal of Microsoft Windows sees a carriage return character, it moves the cursor to the beginning of the current line. Hence, the command line "AT+CGMI<CR>" that you entered and the corresponding response will be displayed like this in a terminal program such as HyperTerminal of Microsoft Windows:

AT+CGMI

Nokia

OK

SMS Center

An SMS center (SMSC) is responsible for handling the SMS operations of a wireless network. When an SMS message is sent from a mobile phone, it will reach an SMS center first. The SMS center then forwards the SMS message towards the destination. An SMS message may need to pass through more than one network entity (e.g. SMSC and SMS gateway) before reaching the destination. The main duty of an SMSC is to route SMS messages and regulate the process. If the recipient is unavailable (for example, when the mobile phone is switched off), the SMSC will store the SMS message. It will forward the SMS message when the recipient is available.

GSM MODEM INTERFACING WITH MICROCONTROLLER

GSM modem cannot be interfaced with the microcontroller directly. Thus, serial communication is used to interface modem with the microcontroller.

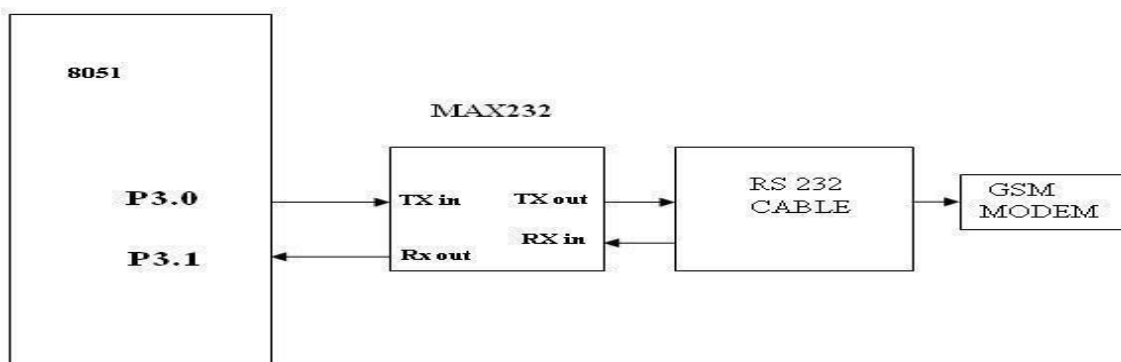


Fig 5.5: Use of Max232

5.5 LM35 Temperature sensor:

- ☐ Transducers convert physical data such as temp, light intensity, speed etc to electrical signal.
- ☐ Depending on the transducer, the output produced in the form of voltage, current, resistance or capacitance.
For example: Temperature is converted in to electrical signals using a transducer called Thermistor. A thermistor responds to the temperature change by changing resistance, but its response is not linear.
- ☐ That's why we are using linear temperature sensors like LM34 or LM35 series. The LM35 series are precision integrated-circuit temperature sensors, whose output voltage is linearly proportional to the Celsius (Centigrade) temperature.
- ☐ Its output is 10 MilliVolts per degree centigrade
- ☐ So if the output is 310 mV then temperature is 31 degree C.
- ☐ It has a range of -55 to $+150^{\circ}\text{C}$ temperature range.
- ☐ LM35 is a popular and low cost temperature sensor.
- ☐ It has three pins as follows.
- ☐ To use the sensor simply connect the Vcc to 5V, GND to Gnd and the Out to one of the ADC (analog to digital converter channel). The output linearly varies with temperature

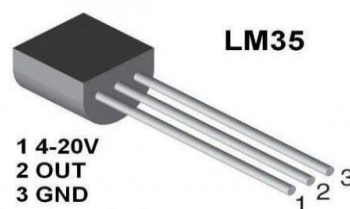


Fig 5.6: LM35 TEMPERATURE SENSOR

5.6 Relay

A Relay driver IC is an electro-magnetic switch that will be used whenever we want to use a low voltage circuit to switch a light bulb ON and OFF which is connected to 220V mains supply.

The required current to run the relay coil is more than can be supplied by various integrated circuits like Op-Amp, etc. Relays have unique properties and are replaced with solid state switches that are stronger than solid-state devices. High current capacities, capability to stand ESD and drive circuit

isolation are the unique properties of Relays. There are various ways to drive relays. Some of the Relay Driver ICs are as be below



Fig 5.7 RELAY

High side toggle switch driver

Low side toggle switch driver

Bipolar NPN transistor driver

N-Channel MOSFET driver and

Darlington transistor driver

ULN2003 driver

Relay Driver IC Circuit

Relays are components that permit a low-power circuit to control signals or to switch high current ON and OFF which should be electrically isolated from controlling circuit.

The Required Components:

1)Zener Diode

2)6-9V Relay

3)9V Battery or DC Power

4)Input Voltage Source

5)Supply 2N2222 Transistor

6)1K Ohm Resistor

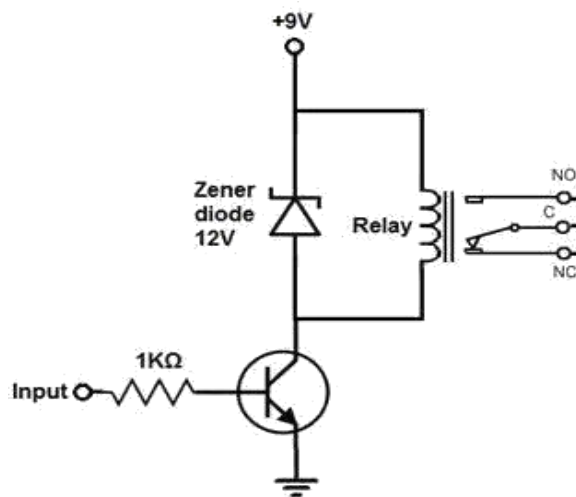


Fig 5.8 RELAY CIRCUIT DIAGRAM

In order to drive the relay, we use transistor and only less power can be possibly used to get the relay driven. Since, transistor is an amplifier so the base lead receives sufficient current to make more current flow from Emitter of Transistor to Collector. If the base once gets power that is sufficient, then the transistor conduct from Emitter to Collector and power the relay.

The Transistor's emitter-to-collector channel will be opened even though no input current or voltage is applied to Base lead of Transistor. Therefore, blocking current flows through relay coil.

The emitter-to-collector channel will be opened and allows current to flow through relay's coil if enough current or voltage is applied as input to the base lead. AC or DC Current can be used to power the relay and circuit. Relays are electromagnetic devices which allow low-power circuit to switch a high current ON and OFF switching devices with the help of an armature that is moved by an electromagnet.

Driver Circuit is used to boost or amplify signals from micro-controllers to control power switches in semi-conductor devices. Driver circuits take functions that include isolating the control circuit and the power circuit, detecting malfunctions, storing and reporting failures to the control system, serving as a precaution against failure, analyzing sensor signals and creating auxiliary voltages.

A negative voltage spike is produced when turning the device OFF. A diode is also connected across the transistor instead of the load sometimes in order to protect the transistor. The 2N3904 shown below is a small discrete BJT transistor is used for a driver circuit that required less than 200mA. In this circuit with BJTs, Vcc – higher voltage supply than the logic power supply and 6 or 12V DC is required for motors or relays.

The load is directly connected to battery power and cannot pass through the voltage regulator in battery operated devices. Many devices such as motors have more inflow current spike when they are first turned ON. Be cautious on maximum current ratings.

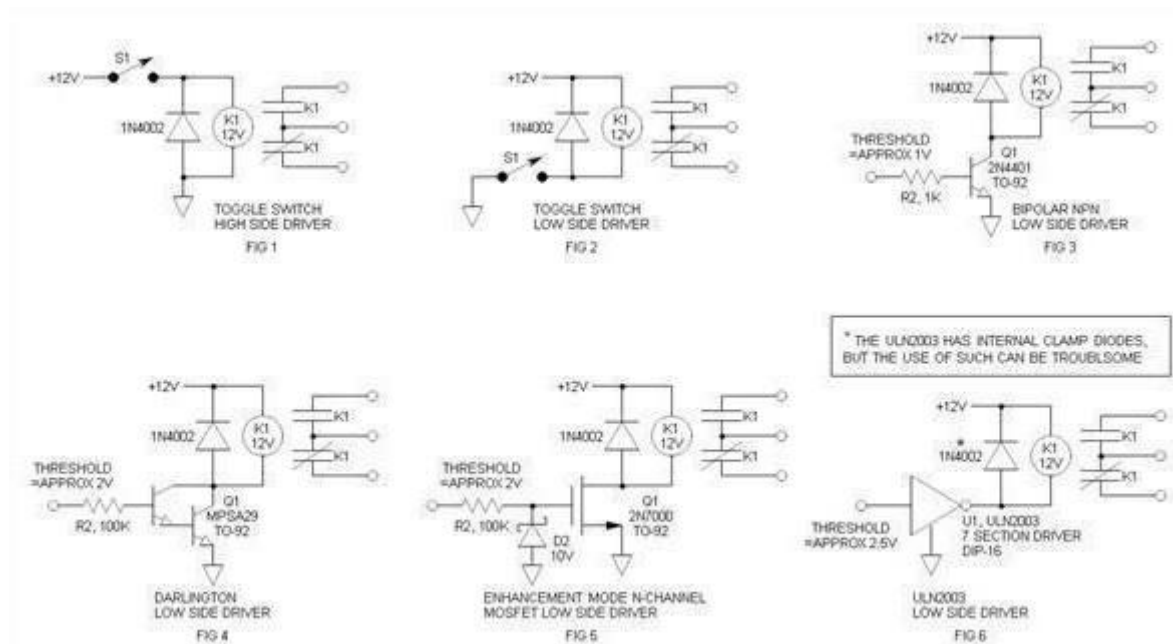


Fig 5.9: internal circuit diagram of relay

Advantages of Low Side Driver:

- 1) More interface options are available which includes popular ULN2003 driver.
- 2) Easy to interface to low voltage logic circuitry.
- 3) Fewer components are used.
- 4) Less expensive NPN drive transistors.
- 5) Relay power reduces load on voltage regulator.
- 6) It uses more commonly obtained NPN drive

transistors. It is easier to interface relay.

- 7) It is economic.
- 8) Uses Industry standard technique.

The ULN2003 has internal clamp diodes. While these work OK in non-critical applications and it leads to rise of glitches.

Clamp Diode

The clamp, free-wheeling or commutation diode provides a path for the inductive discharge current to flow when the driver switch is opened. If not provided, it will generate an arc in the switch—while the arc will not generally damage a switch contact, it will cause contact degradation over time—and yes, it will destroy transistors—been there, done that. The diode requirements are non-critical and a 1N4148 signal diode will generally work OK in low power applications.

Avoid emitter follower drivers. If the relay is switched to OFF in 4007 diode eliminates back e.m.f and safe guards the transistor. ON status of the relay is indicated by LED.

DC Relay Driver IC Circuit

Let us see construction of relay driver circuit for relays that are operated from DC power. In order to drive a DC relay, DC voltage is needed in required quantity to rate a relay and a zener diode. Voltage is required for the relay to operate and to open or close its switch in a circuit.

Relays exist with a voltage rating. This is known as relay's datasheet to rate its coil voltage. For the function of relay, it must receive this voltage at its coil terminals. Thus, if a relay has a rated voltage of 9VDC, it should get 9 volts of DC voltage for its working. In order to eliminate voltage spikes from a relay circuit, a diode is required for its proper functioning. The coil of a relay acts an Inductor.

The inductors are electronic components which withstand changes in current and also the inductors are coils of wires wrapped around a conductive core. Voltage spikes damages all components in a circuit and also damages relay's switch contacts. To prevent these voltage spikes, a diode is kept reverse biased in parallel with the relay which acts as a transient (spike) suppressor eliminates voltage spikes by going into conduction before voltage is formed across the coil. A transient suppressor suppresses these spikes. A diode conducts reverse bias current if voltage reaches a certain threshold. The diode functions to shunt excess power to ground, and the diodes conduct if the voltage reaches breakdown voltage.

The Required Components

1)DC Relay

2) Zener Diode

3) DC Voltage Source or a DC power supply.

4) The zener diode is placed reverse biased in parallel to the relay.

The Relay used in the above is rated for 9Volts. In this a 9V DC Voltage source feeds the resistor. A Zener diode reverse biased is placed in order to suppress the transients caused by opening and closing the relay. This shunts all excess power to ground if it reaches a particular threshold. This is the process to operate a relay. Driving the loads which were connected to the output taking required power the relay will be closed.

AC Relay Driver IC Circuit

This AC Relay driver IC circuit is a relay that runs with AC power and cannot be run with DC power. In order to run an AC relay, enough AC voltage is required to rate the relay and transient suppressor. In AC relay circuit we cannot use a diode to remove voltage spikes. This diode conducts an alternate half-cycle with AC power. We use an RC series network by placing across coil in parallel to form a working transient voltage suppressor with an AC circuit. Capacitor absorbs charge which comes excessively and resistor helps to control overflow. Components required to form the circuit is as follows

AC Relay

100 Ohm Resistor

0.05 Micro Farad Capacitor

AC Voltage Source

NOTE: AC voltage source may come out from plug that is inserted into US wall outlet.

Be careful with AC Power that comes out directly from wall outlet as it causes Shock. Consult a Professional before taking power from plug into wall outlet.

When we use a relay with rated voltage 110VAC, we should feed it with 110V from an AC power source. To suppress voltage spikes, resistor and capacitor connected in series acts as transient voltage suppressor.

5.6.1 Relay Driver IC ULN2003

The relay driver uln2003 ic is a high voltage and current Darlington array ic, it comprises of 7-open collector Darlington pairs with common emitters. A pair of Darlington is an arrangement of two bipolar transistors. This IC belongs to the family of ULN200x ICs and various types of this family interface to various logic families. This ULN2003 IC is for 5V TTL and CMOS. logic devices. These ICs are used as relay drivers as well as to drive a wide range of loads, line drivers, display drivers etc. This IC is also normally used while driving Stepper Motors. The pairs of Darlington in ULN2003 is esteemed at 500mA and can withstand peak current of 600mA. In the pin layout, the i/ps & o/ps are provided reverse to each other. Each driver also has a suppression diode to dissipate voltage spikes while driving inductive loads.

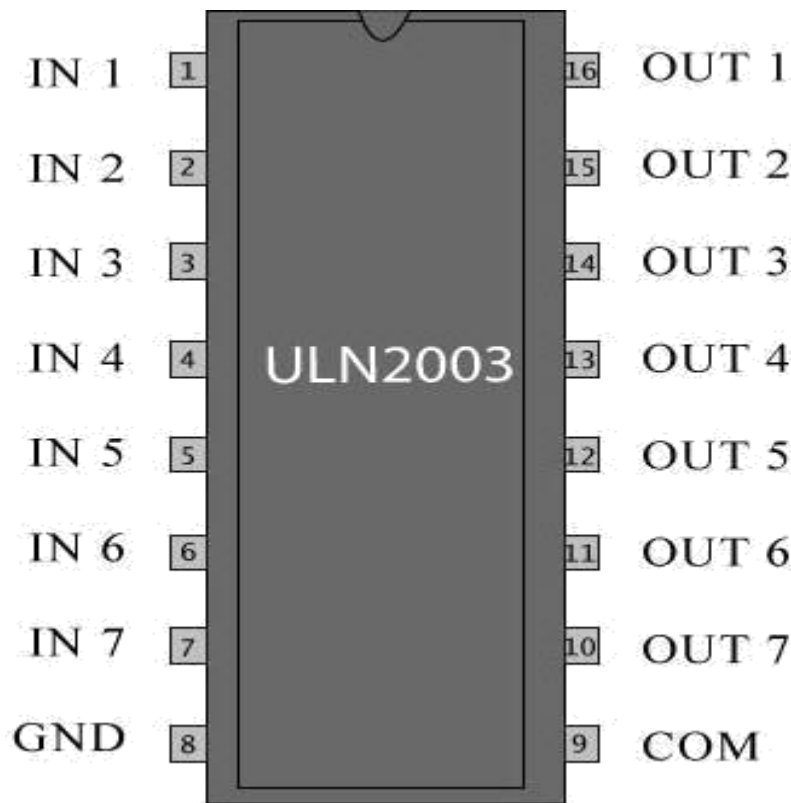


Fig 5.10 ULN 2003

This project is designed for a three-phase-solid-state relay system. It incorporates three single-phase units wherein each phase is controlled individually by power TRIAC with RC snubber network for a zero-voltage switching (ZVS).

Opto-isolators are used in each phase to receive switching signals from a microcontroller of the 8051 family; loads are connected in series with a set of TRIACS driven by an opto-isolator. The microcontroller is designed to generate output pulses after zero voltage pulse to ensure that the load gets switched on at zero cross of the supply waveform.

The zero crossing feature of the TRIAC driver, (an opto-isolator) ensures low noise generation thus avoiding sudden inrush of current on resistive and inductive loads. In this project, two push buttons are used for generating the output pulses from the microcontroller randomly, away from the ZVS ie not coinciding with zero voltage supply voltage of the waveform.

5.7Soil Moisture Sensors:

Soil moisture sensors measure the volumetric water content in soil. Since the direct gravimetric measurement of free soil moisture requires removing, drying, and weighting of a sample,

soil moisture sensors measure the volumetric water content indirectly by using some other property of the soil, such as electrical resistance, dielectric constant, or interaction with neutrons, as a proxy for the moisture content. The relation between the measured property and soil moisture must be calibrated and may vary depending on environmental factors such as soil type, temperature, or electric conductivity. Reflected microwave radiation is affected by the soil moisture and is used for

sensing in hydrology and agriculture. Portable probe instruments can be used by farmers or gardeners.

Soil moisture sensors typically refer to sensors that estimate volumetric water content. Another class of sensors measure another property of moisture in soils called water potential; these sensors are usually referred to as soil water potential sensors and include torsionmeter's and gypsum blocks.

5.7.1 Sensor Technology:

Technologies commonly used to indirectly measure volumetric water content (soil moisture)

include)

- Frequency Domain Reflectometry (FDR): The dielectric constant of a certain volume element around the sensor is obtained by measuring the operating frequency of an oscillating circuit.
- Time Domain Transmission (TDT) and Time Domain Reflectometry (TDR): The dielectric constant of a certain volume element around the sensor is obtained by measuring the speed of propagation along a buried transmission line.
- Neutron moisture gauges: The moderator properties of water for neutrons are utilized to estimate soil moisture content between a source and detector probe.
- Soil resistivity: Measuring how strongly the soil resists the flow of electricity between two electrodes can be used to determine the soil moisture content.
- Galvanic cell: The amount of water present can be determined based on the voltage the soil produces because water acts as an electrolyte and produces electricity. The technology behind this concept is the galvanic cell.

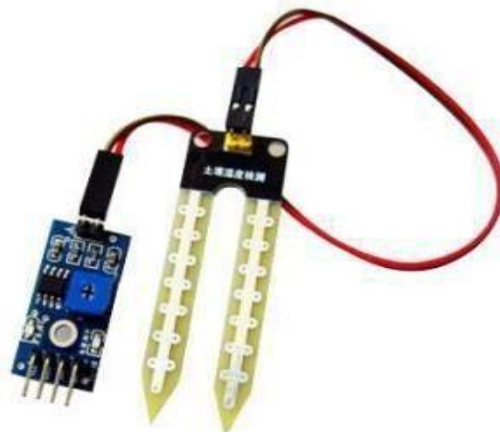


FIG5.11: SOIL SENSOR

5.8 Liquid Crystal Display:

LCD stands for **L**iquid **C**rystal **D**isplay. LCD is finding wide spread use replacing LEDs (seven segment LEDs or other multi segment LEDs) because of the following reasons:

1. The ability to display numbers, characters and graphics. This is in contrast to LEDs¹. The declining prices of LCDs.
2. , which are limited to numbers and a few characters.

3. Incorporation of a refreshing controller into the LCD, thereby relieving the CPU of the task of refreshing the LCD. In contrast, the LED must be refreshed by the CPU to keep displaying the data.
4. Ease of programming for characters and graphics.

These components are “specialized” for being used with the microcontrollers, which means that they cannot be activated by standard IC circuits. They are used for writing different messages on a miniature LCD.

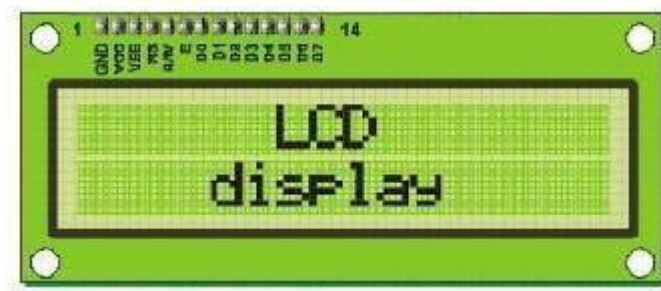


Fig 5.12: LCD Display

A model described here is for its low price and great possibilities most frequently used in practice. It is based on the HD44780 microcontroller (*Hitachi*) and can display messages in two lines with 16 characters each. Automatic shifting message on display (shift left and right), appearance of the pointer, backlight etc. are considered as useful characteristics.

5.8.1 Pins Functions

There are total of 14 pins marked with numbers (16 in case the background light is built in).

Their function is described in the table below:

Function	Pin Number	Name	Logic	Description
Ground	1	VSS	-	0v
Power Supply	2	VDD	-	+5v
Contrast	3	VEE	-	0-VDD
	4	RS	0	D0-D7 are interpreted as commands
			1	D0-D7 are interpreted as data
Control of operating	5	R/W	0	Write data(from controller to LCD)
			1	Read data(from LCD to controller)
	6	E	0	Acess to LCD Disabled
			1	Normal Operating
Data Commands	7	D0	0	From 1 to Data commands are transferred to LCD
			0/1	Bit 0 LSB
	8	D1	0/1	Bit 1
	9	D2	0/1	Bit 2
	10	D3	0/1	Bit 3

	11	D4	0/1	Bit 4
	12	D5	0/1	Bit 5
	13	D6	0/1	Bit 6
	14	D7	0/1	Bit 7 MSB

TABLE 5.3 PIN CONFIGURATION

5.8.2LCD screen:

LCD screen consists of two lines with 16 characters each. Each character consists of 5x7 dot matrix. Contrast on display depends on the power supply voltage and whether messages are displayed in one or two lines. For that reason, variable voltage 0-V_{dd} is applied on pin marked as V_{ee}. Trimmer potentiometer is usually used for that purpose. Some versions of displays have built in backlight (blue or green diodes).

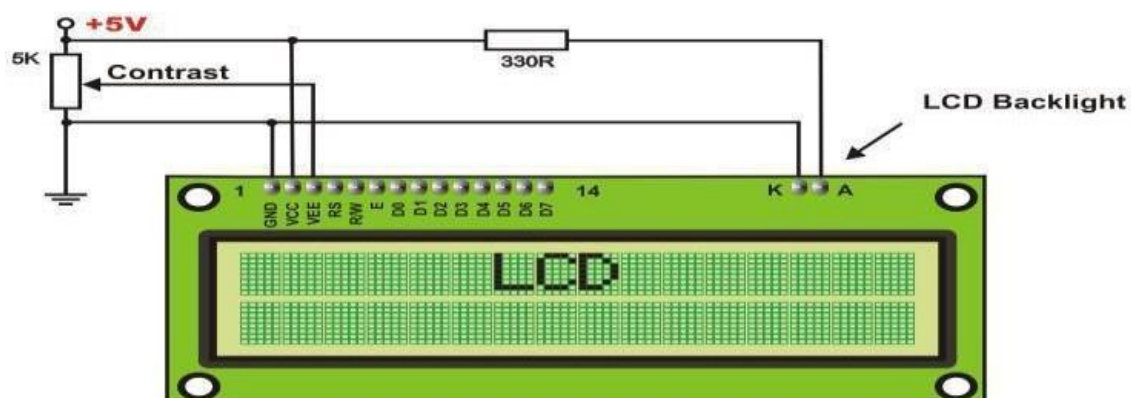


Fig 5.13: LCD Connection

5.8.3LCD Basic Commands

All data transferred to LCD through outputs D0-D7 will be interpreted as commands or as data, which depends on logic state on pin RS:

RS = 1 - Bits D0 - D7 are addresses of characters that should be displayed. Built in processor addresses built in “map of characters” and displays corresponding symbols. Displaying position is determined by DDRAM address. This address is either previously defined or the address of previously transferred character is automatically incremented.

RS = 0 - Bits D0 - D7 are commands which determine display mode. List of commands which

LCD recognizes are given in the table below:

Command	Execution											Time	
	RS	RW	D7	D6	D5	D4	D3	D2	D1	D0			
Clear display			0	0		0	0	0	0	0	0	1	1.64Ms
Cursor home			0	0		0	0	0	0	0	1	x	1.64mS
Entry mode set			0	0	0	0	0	0	0	1		I/D S	40uS
Display on/off control			0	0	0	0	0	0	1	D	U	B	40uS
Cursor/Display Shift			0	0	0	0	0	1	D/C	R/L	x	x	40uS
Function set			0	0	0	0	1	DL	N	F	x	x	40uS
Set CGRAM address			0	0	0	1	CGRAM address					40uS	
Set DDRAM address			0	0	1	DDRAM address					40uS		
Read “BUSY” flag (BF)			0	1	BF DDRAM address					-			
Write to CGRAM or DDRAM	1	0	D7	D6	D5	D4	D3	D2	D1	D0	40uS		
Read from CGRAM or DDRAM			1	1	D7	D6	D5	D4	D3	D2	D1	D0	40uS
I/D 1 = Increment (by 1) 0 = Decrement (by 1)													
R/L 1 = Shift right 0 = Shift left													
S 1 = Display shift on 0 = Display shift off													
DL 1 = 8-bit interface 0 = 4-bit interface													
D 1 = Display on 0 = Display off													
N 1 = Display in two lines 0 = Display in one line													
U 1 = Cursor on 0 = Cursor off													
F 1 = Character format 5x10 dots 0 = Character format 5x7 dots													
B 1 = Cursor blink on 0 = Cursor blink off													
D/C 1 = Display shift 0 = Cursor shift													

Table 5.4: LCDCCommands

5.8.4LCD Connection

Depending on how many lines are used for connection to the microcontroller, there are 8-bit and 4-bit LCD modes. The appropriate mode is determined at the beginning of the process in a phase called “initialization”. Consequently, each data is sent to LCD in two steps: four higher bits are sent first (that normally would be sent through lines D4-D7), four lower bits are sent afterwards. Besides, with regards to the fact that data are rarely read from LCD (data mainly are transferred from microcontroller to LCD) one more I/O pin may be saved by simple connecting R/W pin to the Ground. Such saving has its price.

Even though message displaying will be normally performed, it will not be possible to read from busy flag since it is not possible to read from display.

5.8.5LCD Initialization

Once the power supply is turned on, LCD is automatically cleared. This process lasts for approximately 15mS. After that, display is ready to operate. The mode of operating is set by default. This means that:

1. Mode

DL = 1 Communication through 8-bit interface N =

0 Messages are displayed in one line

F = 0 Character font 5 x 8 dots

2. Display/Cursor off D = 0

Display off

U = 0 Cursor off

B = 0 Cursor blink off

3. Character entry

ID = 1 Addresses on display are automatically incremented by 1 S = 0

Display shift off

Automatic reset is mainly performed without any problems. If for any reason power supply voltage does not reach full value in the course of 10mS, display will start perform completely unpredictably?

5.9Resistive Humidity Sensor:

Humidity Sensor is one of the most important devices that has been widely in consumer, industrial, biomedical, and environmental etc. applications for measuring and monitoring Humidity.

Humidity is defined as the amount of water present in the surrounding air. This water content in the air is a key factor in the wellness of mankind. For example, we will feel comfortable even if the temperature is 00C with less humidity i.e. the air is dry.

But if the temperature is 100C and the humidity is high i.e. the water content of air is high, then we will feel quite uncomfortable. Humidity is also a major factor for operating sensitive equipment like electronics, industrial equipment, electrostatic sensitive devices and high voltage devices etc. Such sensitive equipment must be operated in a humidity environment that is suitable for the device.

5.9.1 Working of resistive humidity sensor:

The Resistive Humidity Sensor is usually made up of materials with relatively low resistivity and this resistivity changes significantly with changes in humidity. The relationship between resistance and humidity is inverse exponential. The low resistivity material is deposited on top of two electrodes.

The electrodes are placed in interdigitated pattern to increase the contact area. The resistivity between the electrodes changes when the top layer absorbs water and this change can be measured with the help of a simple electric circuit.

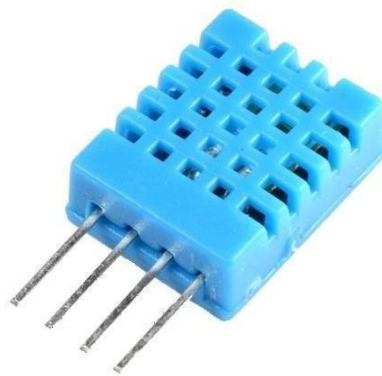


Fig 5.14 Resistive Humidity sensor

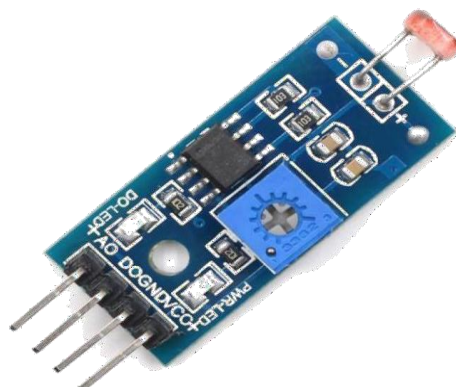
5.10 Light Sensor:

Light sensors are **a type of photo detector (also called photo sensors) that detect light**. Different types of light sensors can be used to measure illuminance, respond to changes in the amount of light received, or convert light to electricity.

Light sensor will send the light and set an optimum temperature in the greenhouse for the plants.

An artificial light will be switched on automatically by the light sensor with the help of a relay and also switched off automatically.

The light sensor is involved to resemble the action of sunlight



CHAPTER 6

SOFTWARE MODULE

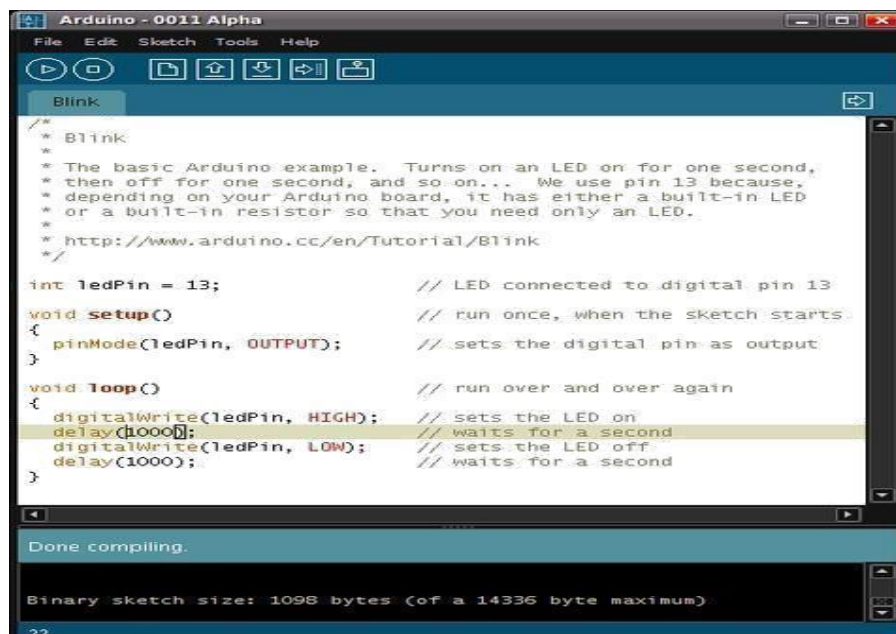
6.1S oftw are:

The software used by the arduino is Arduino IDE.

The Arduino IDE is a cross-platform application written in Java, and is derived from the IDE for the Processing programming language and the Wiring project. It is designed to introduce programming to artists and other newcomers unfamiliar with software development. It includes a code editor with features such as syntax highlighting, brace matching, and automatic indentation, and is also capable of compiling and uploading programs to the board with a single click. There is typically no need to edit make files or run programs on a command. Although building on command-line is possible if required with some third-party tools such as Ino.

The Arduino IDE comes with a C/C++ library called "Wiring" (from the project of the same name), which makes many common input/output operations much easier. Arduino programs are written in C/C++, although users only need define two functions to make a runnable program:

- `setup()` – a function run once at the start of a program that can initialize settings
- `loop()` – a function called repeatedly until the board powers off



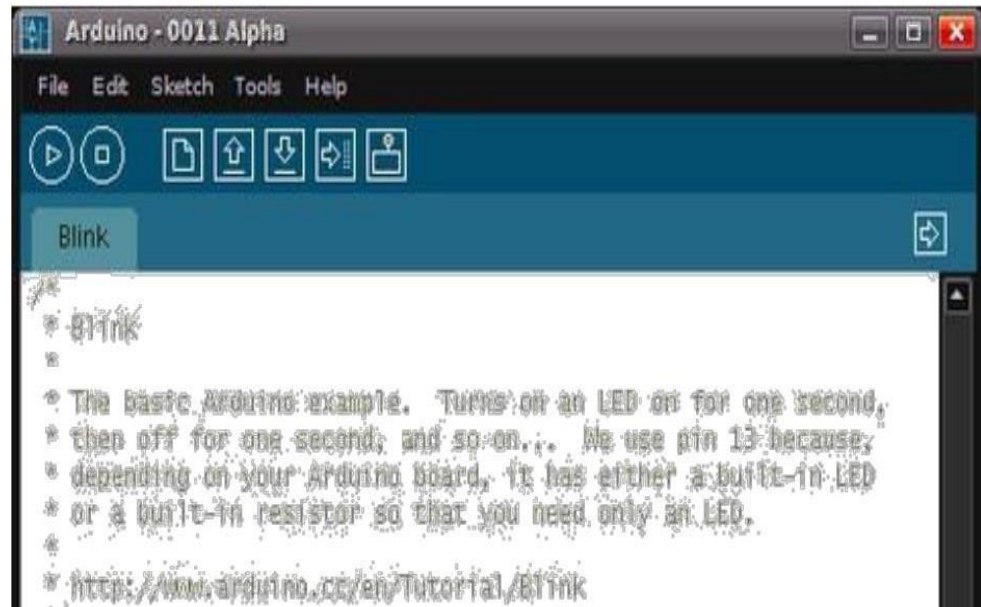


Figure 6.1 A screenshot of the Arduino IDE showing the "Blink" program, a simple beginner program

A typical first program for a microcontroller simply blinks a LED on and off. In the Arduino environment, the user might write a program like this:

```
#define LED_PIN 13

void setup () {
    pin Mode (LED_PIN, OUTPUT); // enable pin 13 for digital output
}

void loop () {
    digital Write (LED_PIN, HIGH); // turn on the LED
    delay (1000);                // wait one second (1000 milliseconds)
    digital Write (LED_PIN, LOW); // turn off the LED
    delay (1000);                // wait one second
}
```

For the above code to work correctly, the positive side of the LED must be connected to pin 13 and the negative side of the LED must be connected to ground. The above code would not be seen by a standard C++ compiler as a valid program, so when the user clicks the "Upload to I/O board" button in the IDE, a copy of the code is written to a temporary file with an extra include header at the top and a very simple [main\(\) function](#) at the bottom, to make it a valid C++ program.

The Arduino IDE uses the [GNU toolchain](#) and [AVR Lab](#) to compile programs, and uses [avrdude](#) to upload programs to the board.

For educational purposes there is third party graphical development environment called [Miniblog](#) available under a different open source license.

language reference

Arduino programs can be divided in three main parts: *structure*, *values* (variables and constants), and *functions*.

Available data types in ARDUINO IDE are

- void
- Boolean
- char (0 – 255)
- byte - **8-bit data (0 – 255)**
- int - **16-bit data (32,767 - -32,768)**
- long – **32-bit data (2,147,483,647 to -2,147,483,648)**
- float
- double
- string - char array
- String - object
- array

Arithmetic operators

Arithmetic operators include addition, subtraction, multiplication and division. For math that requires fractions, you can use float variables, if you can bear large size and slow computation speeds in your microcontroller.

e.g.,

```
y = y + 3;
```

```
x = x - 7;
```

```
i = j * 6;
```

```
r = r / 5;
```

Comparison operators

Comparisons of one variable or constant against another are often used in if statements to test if a specified condition is true.

e.g.,

```
x == y // x is equal to y
```

```
x != y // x is not equal to y
```

```
x < y // x is less than y
```

```
x > y // x is greater than y
```

```
x <= y // x is less than or equal to y
```

```
x >= y // x is greater than or equal to y
```

Logical operators

Logical operators are usually a way to logically combine two expressions and return a **TRUE** or **FALSE** depending on the operator.

There are three logical operators, **AND, OR, and NOT**.

e.g.,

Logical AND:

```
if (x > 0 && x < 5) // true only if both expressions are true
```

Logical OR:

```
if (x > 0 || y > 0) // true if either expression is true
```

Logical NOT:

```
if (! x > 0) // true only if expression
```

TRUE/FALSE

These are Boolean constants that define logic levels of the arduino.

FALSE is easily defined as 0 (zero)

TRUE is often defined as 1, but can also be anything else except zero. So in a Boolean sense, -1, 2, and -200 are all also defined as TRUE.

e.g.,

```
if (abcd== TRUE);
```

```
{  
DoSomethingNice;  
}  
else  
{  
DoSomethingHorrible;  
}
```

HIGH/LOW

These constants define pin levels as HIGH or LOW and are used when reading or writing to digital pins.

HIGH is defined as logic level 1, ON, or 5 volts

LOW is logic level 0, OFF, or 0 volts.

e.g.,

```
digital Write (13, HIGH);
```

INPUT/OUTPUT

These constants define pin levels as HIGH or LOW and are used when reading or writing to digital pins.

HIGH is defined as logic level 1, ON, or 5 volts

LOW is logic level 0, OFF, or 0 volts.

e.g.,

```
pin mode (13, OUTPUT);
```

Arduino/processing language Comparison

The Arduino language (based on Wiring) is implemented in C/C++, and therefore has some differences from the Processing language, which is based on Java.

Arrays	
Arduino	Processing
<code>int bar[8]; bar[0] = 1;</code>	<code>int[] bar = new int[8]; bar[0] = 1;</code>
<code>int foo[] = { 0, 1, 2 };</code>	<code>int foo[] = { 0, 1, 2 };</code> or <code>int[] foo = { 0, 1, 2 };</code>
Loops	
Arduino	Processing
<code>int i; for (i = 0; i < 5; i++) { ... }</code>	<code>for (int i = 0; i < 5; i++) { ... }</code>

Fig6.2 Comparison between arduino and processing language

SIMULATOR for ARDUINO:

The Arduino Simulator

app gives the user the freedom to work without the basic setup of hardware and

software. It is designed to be used by beginners and also, experienced developers, who want to quickly develop Arduino projects.

The developer can make the necessary changes in the code - delay, pin number, and state - 0 (low) 1 (high) - and check it immediately. The app shows the breadboard, complete with 14 LED pins.

- You can drag and place the wires in the correct positions to connect to Arduino. If the wires are placed according to the code, then it will show the expected results. Once
- satisfied, you can save it and email it. The code can be copied and used in an actual project just as easily.
- This app is an easy way to work through Arduino projects. With customisable codes, and a simple to use interface, this Arduino Simulator app from Schogini Systems is a convenient app for Arduino developers.

A screenshot of Arduino simulator is shown in the figure below

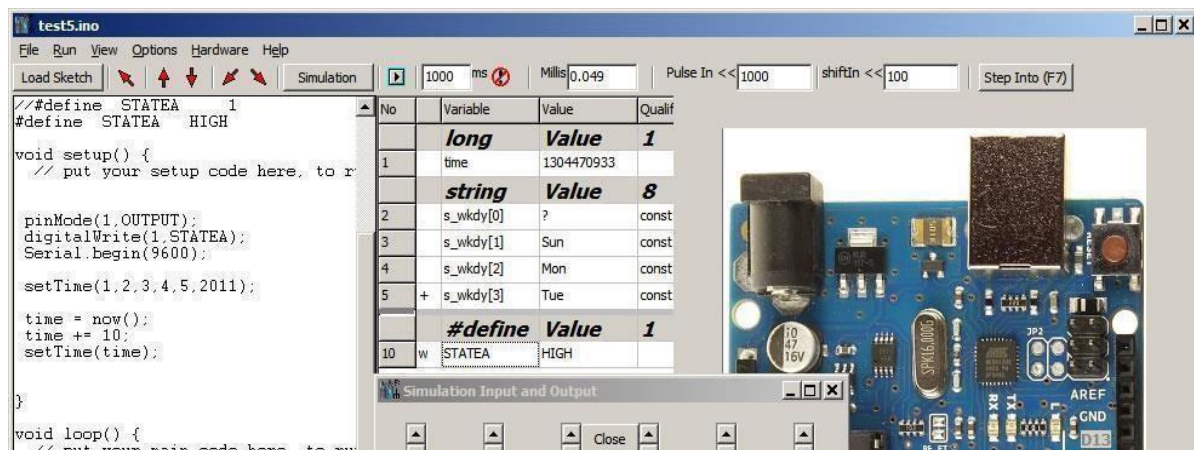


Figure 6.3 screenshot of ARDUINO simulator

APPLICATIONS OF ARDUINO:

Arduino was basically designs to make the process of using electronics in multidisciplinary projects more accessible. It is intended for artists, designers, hobbyists, and anyone interested in creating interactive objects or environments. Arduino can sense the environment by receiving input from a variety of sensors and can affect its surroundings by controlling lights, motors, and other actuators. Because of these features, Arduino finds extensive application in various fields. Arduino projects can be stand-alone or they can communicate with software running on a computer.

Arduino received an Honorary Mention in Digital Communication section of the 2006 Ars Electronica Prix

Arduino is used by all class of people in a different way. Some students use it in their projects, some using Arduino for fun, some went out to become entrepreneurs. This only shows how useful is this tiny device.

ARDUINO is spreading rapidly across the globe. Arduino is actually an open source hardware project that can be programmed to read temperatures, control a motor, and sense touch.

the Arduino is both a cute, blue micro controller platform that fits nicely in the palm of your hand and an expanding community of developers who support it, distributed across two dozen countries, four continents, and counting.

The Arduino board is for anyone who wants to build a basic level of intelligence into an object. Once programmed, it can read sensors, make simple decisions, and control myriad devices in the real world. Using it is a snap: first, hook up a few sensors and output devices to the Arduino, then program it using the free developer's software. Next, debug your code and disconnect the Arduino. Then, the little blue Arduino becomes a standalone computer.

The original intention of the Arduino project was to see what would happen if community support were substituted for the corporate support that is usually required for electronics development. The first developers — Massimo Benzie, David Cuatrillos, David Mallis, and

Nicholas Gambetta — ran a series of workshops on assembling the Arduino, giving away the board to stimulate development.

CHAPTER 7

SOURCE CODE

```
#include <LiquidCrystal.h>
```

```
LiquidCrystal lcd(8,9,10,11,12,13);
```

```
const char* ssid = "project";
```

```
const char* password = "project123";
```

```
char res[130];
```

```
#include<dht.h>
```

```
#define dht_dpin A0
```

```
dht DHT;
```

```
int temperature, humidity,Temp,fire1;
```

```
const int soil=15;
```

```
const int light=16;
```

```
const int fan=5;
```

```
int val;
```

```
int TEMP;
```

```
int gas;
```

```
int TEMP1;
```

```
void serialFlush(){
```

```
while(Serial.available() > 0) {
```

```
    char t = Serial.read();
```

```
}
```

```
}
```

```
int hbeat=0,tempe=0,hum=0,ecg=0,raj=0;
```

```
char check(char* ex,int timeout)
```

```
{
```

```
    int i=0;
```

```
    int j = 0,k=0;
```

```
    while (1)
```

```
    {
```

```
        sl:
```

```
if(Serial.available() > 0)

{

    res[i] = Serial.read();

    if(res[i] == 0x0a || res[i]=='>' || i == 100)

    {

        i++;

        res[i] = 0;break;

    }

    i++;

}

j++;

if(j == 30000)

{

    k++;

    //Serial.println("kk");

    j = 0;
```

```
}
```

```
if(k > timeout)
```

```
{
```

```
    // Serial.println("timeout");
```

```
    return 1;
```

```
}
```

```
}//while 1
```

```
if(!strncmp(ex,res,strlen(ex)))
```

```
{
```

```
    //Serial.println("ok..");
```

```
    return 0;
```

```
}
```

```
else
```

```
{
```

```
    // Serial.print("Wrong ");
```

```
    // Serial.println(res);
```

i=0;

goto sl;

}

}

char buff[200];

void upload1();

void setup() {

int i=0;

char ret;

pinMode(soil,INPUT);

pinMode(light,INPUT);

Serial.begin(9600);

lcd.begin(16,2);

lcd.clear();lcd.setCursor(0, 0);lcd.print("WELCOME");

delay(3000);

serialFlush();

st:

Serial.println("ATE0");

ret = check((char*)"OK",50);

Serial.println("AT");

ret = check((char*)"OK",50);

if(ret != 0)

{

delay(100);

delay(100);

goto st;

}

delay(1000);

Serial.println("AT+CWMODE = 1");

ret = check((char*)"OK",50);

lcd.clear();lcd.setCursor(0, 0);lcd.print("CONNECTING");

cagain:

delay(1000);

serialFlush();

Serial.print("AT+CWJAP=\"");

Serial.print(ssid);

Serial.print("\",\");

Serial.print(password);

Serial.println("\");

if(check((char*)"OK",300))

{

lcd.clear();lcd.setCursor(0, 0);lcd.print("CONNECTING");

goto cagain;

}

delay(1000);

lcd.clear();lcd.setCursor(0, 0);lcd.print("CONNECTED");

Serial.println("AT+CIPMUX=1");

```
delay(1000);delay(1000);
```

```
lcd.clear();
```

```
}
```

```
unsigned long int duration = 0;
```

```
int upload=0,count=0;
```

```
void loop() {
```

```
{
```

```
DHT.read11(dht_dpin);
```

```
lcd.clear();
```

```
lcd.setCursor(0,0);
```

```
lcd.print("H=");
```

```
lcd.print(humidity=DHT.humidity);
```

```
lcd.print(" % ");
```

```
lcd.setCursor(6,0);
```

```
lcd.print("T=");
```

```
lcd.print(temperature=DHT.temperature);
```

```
// lcd.write(1);
```

```
//lcd.print("C");
```

```
delay(100);
```

```
//val = analogRead(1);float T=( val/1024.0)*5000; TEMP= T/10;
```

```
//lcd.setCursor(0,0);lcd.print("TEMP:");lcd.setCursor(6,0);lcd.print(TEMP);delay(50);
```

```
/* gas = analogRead(0); TEMP1=( gas/1024.0)*5000;
```

```
lcd.setCursor(11,0);lcd.print("G=");//lcd.setCursor(6,1); lcd.print(TEMP1);delay(50);*/
```

```
if(digitalRead(soil) == 0)
```

```
{
```

```
lcd.setCursor(0, 1);lcd.print("WET LAND");
```

```
ecg = 1;
```

```
upload = 1;
```

```
}
```

```
else
```

```
{
```

```
ecg = 0;
```

```
}
```

```
delay(500);
```

```
upload = 1;
```

```
}
```

```
if(digitalRead(light) == 0)
```

```
{
```

```
  lcd.setCursor(0, 1);lcd.print("LIGHT DETECTED");
```

```
  fire1 = 1;
```

```
  upload = 1;
```

```
}
```

```
else
```

```
{
```

```
  fire1 = 0;
```

```
}
```

```
count++;
```

```
if(upload == 1||count == 2)
```

```
{
```

```
count =0;
```

```
upload = 0;
```

```
upload1();
```

```
}
```

```
}
```

```
void upload1()
```

```
{
```

```
delay(100);
```

```
// lcd.setCursor(0, 1);lcd.print("UPLOADING"); serialFlush();
```

```
Serial.println("AT+CIPSTART=4,\"TCP\", \"184.106.153.149\",80");
```

```
// if(!check((char*)"Linked",200))
```

```
{

    delay(1000);

    serialFlush();

    Serial.println("AT+CIPSEND=4,100");

    // if(!check((char*)">",50))

    {

        delay(500);

        serialFlush();

        Serial.print("GET /update?api_key=OJTYMTVMAJIQBHXV&");

        sprintf(buff,"&field1=%04u",temperature); Serial.print(buff);

        sprintf(buff,"&field2=%04u",humidity);

        Serial.print(buff);

        sprintf(buff,"&field3=%04u",ecg);

        Serial.print(buff);

        sprintf(buff,"&field4=%04u",fire1);
```

```
Serial.print(buff);
```

```
Serial.println("");
```

```
// if(!check((char*)"OK",200))
```

```
{
```

```
    delay(500);
```

```
    Serial.println("AT+CIPCLOSE");
```

```
}
```

```
}//>
```

```
}//Linke
```

```
}
```

LIMITATIONS

- The proposed system has an accuracy of 80% only, which can be increased by using machine learning process with some additional techniques.
- The designed system can be implemented in a class of strength 40 only, where the shape of classroom must be slant.
- As we give unique eigen value to each student the accuracy of system increases but complexity decreases.
- While giving input of a stranger more possibility to show error as it tries to match the image vector with consisted data vectors.

FUTURE SCOPE

The system we have developed has successfully, able to accomplish the task of marking the attendance in the classroom automatically and output is obtained in an excel sheet as desired in real-time. However, in order to develop a dedicated system which can be implemented in an educational institution, a very efficient algorithm which is insensitive to the lighting conditions of the classroom has to be developed. Also, a camera of the optimum resolution must be utilised in the system. Another important aspect where we can work towards is creating an online database of the attendance and automatic updating of the attendance into it keeping in mind the growing popularity of Internet of Things. This can be done by creating a standalone module which can be installed in the classroom having access to internet, preferably a wireless system. These developments can greatly improve the applications of the project.

CONCLUSION

In this system we have implemented an attendance system for a lecture, section, or laboratory by which lecturer or teaching assistant can record students' attendance. It saves time and effort, especially if it is a lecture with huge number of students. Automated Attendance System has been envisioned for the purpose of reducing the drawbacks in the traditional (manual) system. This attendance system demonstrates the use of image processing techniques in classroom. This system can not only merely help in the attendance system, but also improve the goodwill of an institution.

REFERENCES

- [1] Implementation of “face recognition algorithm for biometrics-based time attendance system”, IJARCET, Volume:6, Issue 8, August 2017, ISSN:2278-7798.
- [2] Brian C. Becker, Enrique G. Ortiz, “Evaluation of Face Recognition Techniques for Application to Facebook”, IJARCET, Volume:162, Issue 18 March 2017, ISSN:0975-8887.
- [3] B. Moghaddam, W. Wahid and A. Pentland. Beyond eigenfaces: probabilistic matching for face recognition. IEEE International Conference on Automatic Face and Gesture Recognition, pages 30-35. 1998
- [4] M. Petrou and P. Bosdogianni. Image Processing: The Fundamentals. John Wiley and Sons, Inc., UK, 2000. 335 pages, ISBN 0-471-99883-4.
- [5] M. Turk and A. Pentland. Eigenfaces for recognition. Journal of Cognitive Neuroscience, 3(1):71-86, 1991.
- [6] H. H. Barret. Foundations of Image Science. John Wiley & Sons, New Jersey, U.K., third edition, 2004.
- [7] Gu G, Perdisci R, Zhang J, Lee W (2008) BotMiner: clustering analysis of network traffic for protocol-and structure-independent Botnet detection. In: USENIX security symposium 2008 Jul 28, vol 5, no 2, pp 139–154

